

CONVEXITY

Trading the Ask: Market Microstructure, Data Floods,
and the Engineering of an Autonomous Options Desk

Findings from building a live AI trading agent on Alpaca's paper market —
why backtests lie, what the SIP/OPRA firehose really looks like,
and how a \$16,030 paper loss became a deterministic risk architecture.

Faruque Amin

Alpaca AI Trading Agents Hackathon · lablab.ai · August 2026

convexity.faruqueamin.com

github.com/faruqueamin/convexity

Preface

This document began as the required one-page hackathon write-up and grew into the thing it was always trying to be: a record of what actually happens when you point an autonomous agent at a live options market. It is written for three audiences at once — the hackathon judges, who need to see that the system is real; engineers building their own agents, who deserve the map of the minefield I did not have; and my future self, who will be tempted to trust a backtest again and needs the reminder.

Everything claimed here is backed by an artifact: the fill ledger, the post-mortem, the gate's journaled verdicts, the brain's journal, the screener's persisted scans. Where the report generalizes beyond the evidence, it says so. The system lost a meaningful amount of paper money in a memorable way; this document is the dividend.

— Faruque Amin, 29 August 2026

Abstract

This report documents the design, failure, and redesign of **Convexity**, an autonomous AI options-trading desk built for the Alpaca AI Trading Agents Hackathon (lablab.ai, 28 August – 4 September 2026). The system scans a ClickHouse tape of ten-second bars for volume breakouts, mirrors those signals into long option positions through Alpaca's paper Trading API, and supervises itself with a deterministic agent layer. Building it against a live market data feed surfaced three classes of problems that most strategy research never meets: **(i)** the bid/ask spread is not a detail — on thin options it is the dominant cost, and any backtest or sizing logic that assumes mid or spot fills is measuring a market that does not exist; **(ii)** consolidated feeds (SIP for equities, OPRA for options) are a firehose — billions of messages per day — that must be funnelled, coalesced and normalized before a single decision can be made; and **(iii)** autonomy amplifies every assumption: on 28 August 2026 the engine lost \$16,030 of paper money in nine minutes by buying 0DTE puts at the ask on indicative quotes 125% wide. The redesign that followed — a mandatory pre-trade RiskGate that re-prices every order at the ask on a fresh quote, an order-lifecycle chase guard, and an always-on agent brain that can only reduce risk — is described in full, with the real fill ledger as evidence.

Keywords: algorithmic trading, market microstructure, bid-ask spread, OPRA, SIP, options market data, autonomous agents, risk gates, paper trading, Alpaca, WebSocket data engineering

Contents

Preface	2
Abstract	3
Contents	4
Notation and Conventions	7
Chapter 1 · Introduction	8
1.1 The hackathon brief	8
1.2 Why options, and why 'Convexity'	8
1.3 Research questions	8
1.4 System in one paragraph	9
1.5 How to read this report	10
1.6 Position within the field	10
Chapter 2 · The Market Is Not the Price on the Screen	11
2.1 Bid, ask, mid: three prices, only two are real	11
2.2 OPRA versus indicative quotes: not all data is the market	11
2.3 ODTE: the same-day-expiry amplifier	12
2.4 Lead market makers and why quotes exist at all	12
2.5 The four Greeks an engineer actually needs	12
2.6 The NBBO and what 'best execution' implies	13
Chapter 3 · The Spot-Pricing Fallacy	14
3.1 The arithmetic of crossing the spread	14
3.2 Where the fiction hid in my own code	14
3.3 Evidence from the fill ledger	15
3.4 Worked example: pricing the RBRK trade honestly	15
3.5 A checklist for honest strategy research	16
Chapter 4 · Drinking from the Firehose: SIP, OPRA, and the Funnel	17
4.1 The scale of the problem	17
4.2 The wanted-set: subscribe to almost nothing	17
4.3 Coalescing: latest-wins, never queue	17
4.4 Normalization: one shape, with provenance	18
4.5 REST as watchdog, not as primary	18

4.6 Capacity arithmetic for one small process	19
4.7 Streaming failure modes and their handling	19
4.8 Protocol notes: the two sockets	19
Chapter 5 · Case Study: Nine Minutes, −\$16,030	20
5.1 Timeline	20
5.2 Root causes, in order of impact	21
5.3 The quote-quality evidence	22
5.4 What the incident cost and what it bought	22
5.5 The ledger, line by line	22
5.6 Counterfactual: the same tape through the new gate	23
Chapter 6 · RiskGate: The Checkpoint That Cannot Be Bypassed	25
6.1 Fresh-quote evaluation at order time	25
6.2 The checks	25
6.3 The loss lock that counts what you stand to lose	26
6.4 One working order per symbol — including DCA	26
6.5 The chase guard	26
6.6 The gate under fire: verdicts as data	27
6.7 Why a gate, and not a library of checks	27
6.8 Configuration is a risk surface	27
6.9 What the gate does not claim to do	28
Chapter 7 · The Order Lifecycle and the Exit Stack	29
7.1 Design principles	29
7.2 Contract selection	29
7.3 Reconciliation	30
7.4 The screener: choosing where to look	30
7.5 A day in the life of a position	30
Chapter 8 · The Agent Layer: Autonomy with an Authority Envelope	32
8.1 The deterministic brain	32
8.2 The LLM agent: prompt-protocol tools	33
8.3 The supervisor: advisory-first	33
8.4 The brain's journal, verbatim	33
8.5 What the LLM is actually for	35

Chapter 9 · Measured Results	36
9.1 Reading the results honestly	36
9.2 Verification, step by step	37
9.3 The desk as an observability surface	37
9.4 What 'working' means now	37
Chapter 10 · Discussion: What Building It Taught	38
10.1 Backtests are hypotheses about execution, not alpha	38
10.2 Agents need authority envelopes, not prompts	38
10.3 The data layer is the strategy's foundation	38
10.4 Limitations	38
10.5 Mapping the work to the judging rubric	38
Chapter 11 · Conclusion and Future Work	40
Future work	40
Open questions	40
References	41
Appendix A — Options Engine Configuration	42
Appendix B — HTTP API Surface	43
Appendix C — Glossary	44
Appendix D — The Public Face	45
Appendix E — Reproduction Guide	48
Appendix F — Repository Map	49
Appendix G — Fill Ledger Field Dictionary	50

Notation and Conventions

Table 1. Conventions used throughout.

Convention	Meaning
\$	US dollars; all account values are Alpaca paper (simulated) funds
Times	US Eastern Time (ET) unless suffixed Z (UTC); the mixed-base bug in the incident ledger is discussed in §4.4
$b \times a$	a two-sided quote: bid b , ask a , in dollars per share
spread %	$(ask - bid) / mid \times 100$, where $mid = (ask + bid) / 2$
mark-to-bid	valuing a long option position at the prevailing bid
at_ask / at_bid / through_bid	ledger fill classes: filled at the quoted ask, at the quoted bid, or beyond the quoted bid
DTE / 0DTE	days to expiry; 0DTE = same-day expiry
OCC code	the standardized option contract identifier (e.g. RBRK 260828P102)
gate / the gate	the RiskGate module unless context says otherwise
the desk	the live web console at /desk; also the system as a whole
-\$16,030	the incident's realized daily loss — written out each time, deliberately

Figures are numbered continuously and are clickable in the PDF viewer's outline; tables likewise. Code excerpts are verbatim from the repository, shortened only by elision of error handling. Where a number comes from the system's own journals or ledger, the source file is named at the point of use; where it comes from the public literature, a superscript reference is given and resolved in the References.

Chapter 1 · Introduction

1.1 The hackathon brief

The Alpaca AI Trading Agents Hackathon set a precise challenge: build an autonomous AI trading agent on Alpaca's developer stack — the Trading API, the MCP server, or the Alpaca CLI — where every strategy must incorporate options trading, developed and tested entirely in Alpaca's paper-trading environment. Judging weighs P&L performance, technology implementation, creativity, and presentation. A fresh paper account funded with \$100,000 of simulated cash is required for judging, so that the judging team can attribute every fill to the submitted agent.

Convexity is my answer: a single Node.js process that detects short-horizon volume breakouts in equities, expresses them as long option positions, manages every position with a deterministic exit stack, and supervises itself with a layered risk intelligence. Everything in this report ran live against the paper market — the fill ledger, the incident, and the fixes are all real artifacts of the system, not illustrations.

1.2 Why options, and why 'Convexity'

A long option position is the cleanest way to buy convexity: the maximum loss is the premium paid — known before entry — while the upside participates in the underlying without a linear cap. For a momentum system that expects occasional large moves, that asymmetry is the whole point. The name is a constant reminder of what the desk is buying: curvature, not direction.

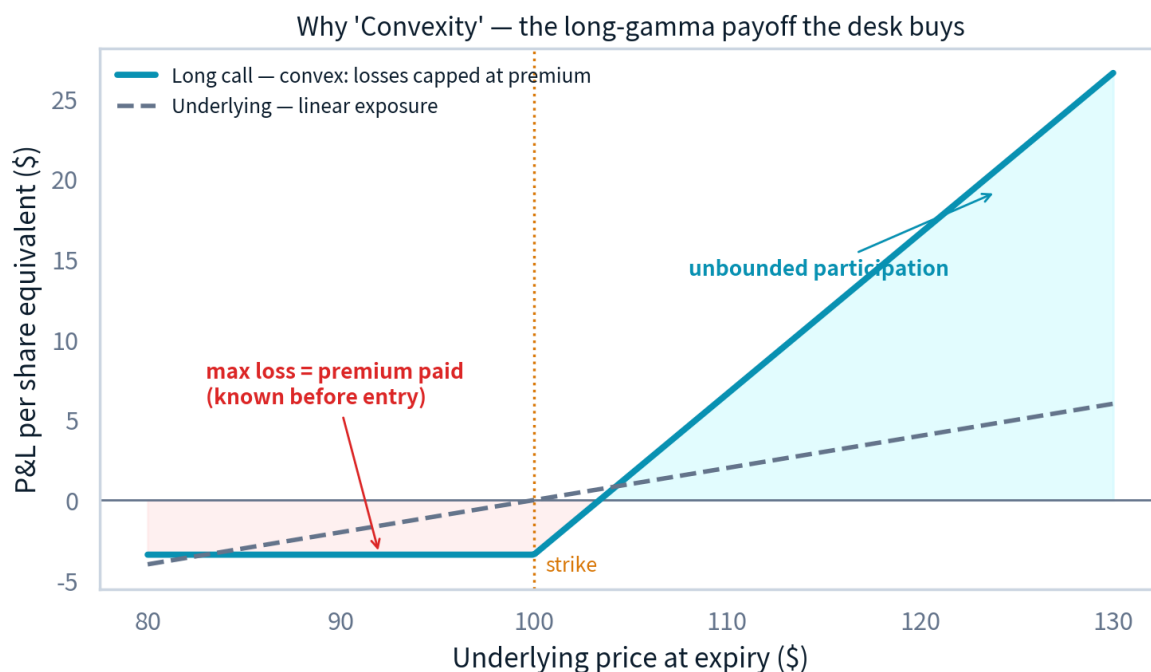


Figure 1. The payoff the desk buys. A long call caps the loss at the premium while participating in upside convexly; the underlying itself is linear. All risk architecture in this report exists to protect the one thing that makes this payoff safe in theory — the premium actually paid — from being much larger than the premium the screen appeared to offer.

1.3 Research questions

The build was treated as an investigation, not just an engineering task. Four questions organize the findings:

- **Q1 — Pricing reality.** What do fills actually cost on the option contracts a small autonomous desk can trade, and how far is that from the spot/mid prices used in typical backtests?
- **Q2 — Data reality.** What does it take to ingest consolidated equity (SIP) and options (OPRA) market data over WebSockets without drowning the decision loop?
- **Q3 — Autonomy risk.** Which failure modes appear only when an agent, not a human, is watching the book?
- **Q4 — Architecture.** What control structure makes an autonomous options desk safe to leave running?

1.4 System in one paragraph

One Node.js process, CommonJS, zero runtime dependencies beyond ws. Market data arrives over two WebSocket streams (SIP equities, OPRA options) through a wanted-set hub that subscribes only to what the desk holds, is working, or watches. Historical context comes from a ClickHouse replica of ten-second to fifteen-minute OHLCV bars and per-ticker metrics. A deterministic signal engine fires volume-breakout entries; the options engine mirrors them into long calls or puts; a mandatory RiskGate re-prices every order on a fresh quote at order time; an exit stack of pure code manages every open position; and an agent brain observes the whole book every fifteen seconds with strictly bounded authority. A local Ollama model provides natural-language reasoning and narration, but can never touch the fill path. Orders go to Alpaca's paper environment; the Alpaca CLI reconciles account state every sixty seconds.

Convexity system architecture — one process, every path gated

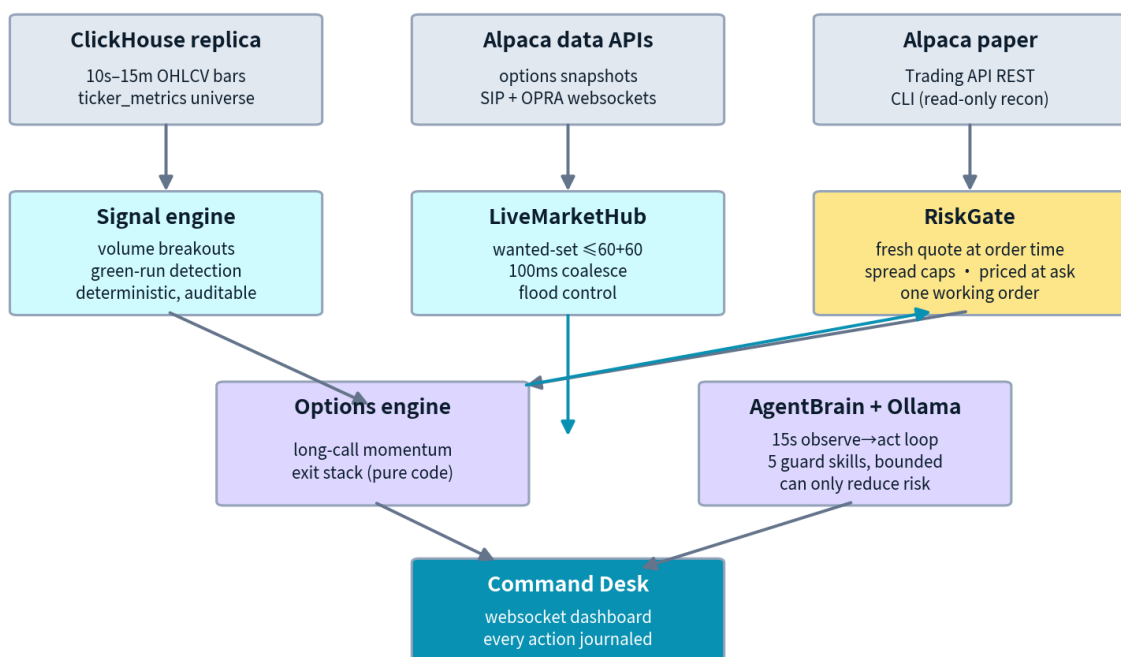


Figure 2. Convexity system architecture. Data flows down from ClickHouse and the Alpaca data APIs; every order path — signal mirror, DCA, agent, or manual — converges on the RiskGate before reaching Alpaca paper trading.

1.5 How to read this report

Chapters 2–4 build the environment: what option prices actually are (2), why the prices used in most research are fiction (3), and what it takes to see the market at all (4). Chapter 5 is the failure that taught the most — reconstructed entirely from the system's own ledger. Chapters 6–8 are the redesign: the gate, the order lifecycle, and the agent layer. Chapter 9 measures; Chapter 10 reflects; Chapter 11 concludes. Readers in a hurry can read the abstract, Chapter 5, and Chapter 6 and come away with the thesis: **the price you can trade at is the only price that matters, and an autonomous system must prove it to itself on every single order.**

1.6 Position within the field

The intellectual debts are market microstructure (Hasbrouck's treatment of execution costs and price discovery^[10]), the option-pricing tradition the local solver descends from (Black–Scholes^[9]), and the operational literature of the feeds themselves (Cboe, Databento, and the SEC's OPRA orders^{[1][2][3]}). What this report adds is not theory but a documented collision: a modern, agent-driven retail stack meeting those realities in production, instrumented well enough that the collision itself became data.

Chapter 2 · The Market Is Not the Price on the Screen

Before the incident can be understood — and before any backtest result can be believed — three market-structure facts have to be internalized. They are elementary to a market maker and routinely invisible to a retail strategy developer. This chapter states them precisely, with the numbers that later chapters rely on.

2.1 Bid, ask, mid: three prices, only two are real

Every option contract carries a **bid** (the best price a buyer is currently willing to pay) and an **ask** (the best price a seller is currently willing to accept). A market order to buy transacts at or near the ask; a market order to sell transacts at or near the bid. The **mid** — $(\text{bid} + \text{ask}) / 2$ — is a computed reference that nobody is obliged to trade at. On liquid underlyings like SPY the spread is cents wide and the distinction is academic. On the long tail of single-name options — exactly where a momentum screener hunts — spreads of 30%, 80%, even 125% of the mid are routine.

The consequence for valuation is brutal: a long position bought at the ask is immediately *marked* at the bid. Buying a contract quoted 0.63×2.77 at the ask means paying \$2.77 for something the market will currently buy back for \$0.63 — an instant -77% mark-to-market loss before the underlying has moved a tick. Section 5 shows this happening with real money (paper money, but real mechanics).

Anatomy of the quote that started the incident — RBRK 260828P102, 09:38:19 ET

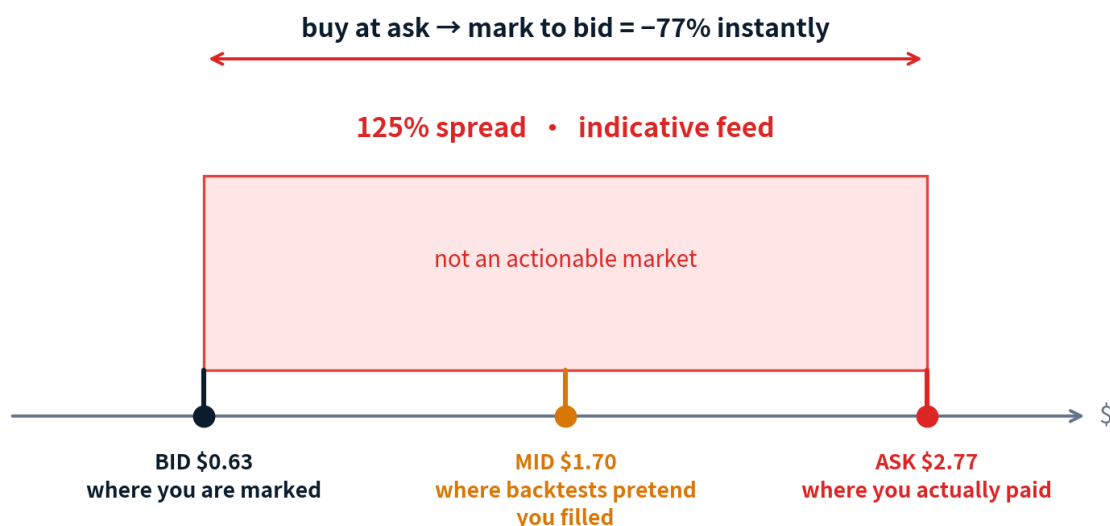


Figure 3. The quote that started the 28 August incident: RBRK 260828P102 quoted 0.63×2.77 on the indicative feed — a 125% spread. The mid (\$1.70) is the price a naive backtest assumes; the ask (\$2.77) is what was actually paid; the bid (\$0.63) is what the position was immediately worth.

2.2 OPRA versus indicative quotes: not all data is the market

US listed options are quoted on eighteen exchanges and consolidated by the Options Price Reporting Authority (OPRA), the securities information processor for options, into a single feed of trades and best bids and offers^[1]. The scale is unlike anything in equities: OPRA reported a peak message volume of **44.8 million messages per second** in Q3 2024, and after its February 2024 migration from 48 to 96 multicast lines the feed's stated capacity rose from 400 billion to one trillion transactions per day^{[1][2]}. During the April 2025 sell-off, peak one-millisecond bursts exceeded 187 million messages per second^[2]. More than a million distinct option contracts are listed and active at any time — two to three orders of magnitude more instruments than the entire US equity market^[2].

Alpaca's options data comes in two tiers. The **OPRA feed** (paid) is the consolidated tape described above. The free **indicative** feed carries delayed/derived values intended to indicate where a contract might trade — it is explicitly not a stream of actionable quotes. An indicative quote can be stale, one-sided, or wider than any real market. A trading system that cannot distinguish the two will, sooner or later, price a decision off a number that nobody would ever trade at. Convexity now tracks feed provenance on every quote and applies a tighter spread cap (15% vs 20%) whenever the feed is indicative — and the incident in Chapter 5 is exactly what happens when it doesn't.

2.3 0DTE: the same-day-expiry amplifier

Zero-days-to-expiry options concentrate everything that is dangerous about option microstructure: extreme gamma (the delta swings violently with the underlying), collapsing theta (the premium decays by the hour), and spreads that are wide in percentage terms even when tight in dollars. A 0DTE contract quoted 0.05×0.10 has a 67% spread on a nickel-wide market. None of this makes 0DTE untradable — it makes it a regime that needs its own, stricter rules: an explicit opt-in flag, a tighter 12% spread cap, and a 50% size haircut in the redesigned gate.

2.4 Lead market makers and why quotes exist at all

Options quotes are mostly machine-made: exchanges appoint Lead Market Makers with quoting obligations across all strikes of an underlying, so quotes update automatically every time the underlying moves — the vast majority of OPRA messages are quote updates, not trades^{[3][5]}. Two practical corollaries. First, quote volume dwarfs trade volume — measured quote updates ran 17.5× stock quote counts on a July 2026 session (6.61 billion options quote updates in one day)^[4]. Second, a quoted ask is not a commitment of size: the displayed size may be a handful of contracts, so a 50-contract order will walk the book. Both facts shaped the ingestion funnel of Chapter 4 and the order sizing of Chapter 6.

2.5 The four Greeks an engineer actually needs

The desk's contract selection and exits lean on four quantities. They are stated here in engineering terms, because each one maps to a concrete gate or exit rung later in the report.

Table 2. The working Greek vocabulary. Values come from the vendor when available; otherwise a local Black-Scholes solver inverts mid + spot into IV and derives the rest — so selection never silently degrades when the free data tier omits greeks.

Greek	Meaning	Where it appears in Convexity
Delta (Δ)	how much the option price moves per \$1 of the underlying	selection floor $ \Delta \geq 0.30$ – 0.40 — deep enough to track the signal, cheap enough to be convex
Gamma (Γ)	how fast delta itself moves — the 'convexity' in the name	the reason ODTE gets its own regime: gamma explodes into expiry
Theta (Θ)	premium decayed per day held	the reason the desk is intraday-only and flattens at 15:50 ET
IV	the market's implied volatility; drives the premium level	the IV-crush exit: -20% IV versus entry IV closes the position

The Black-Scholes fallback deserves emphasis because it illustrates this report's recurring theme. The vendor's free tier frequently omits greeks and IV on exactly the thin contracts the screener surfaces. The pre-incident code treated missing data as 'take the first strike'. The redesign treats missing data as a problem to solve — invert the pricing model locally — and treats unsolvable data as a veto, never as a pass^[9].

2.6 The NBBO and what 'best execution' implies

The quotes this report discusses are the National Best Bid and Offer: the best prices across all exchanges, consolidated by the plan processor. Two implications matter for an autonomous desk. First, the NBBO is a *composite* — the best bid and best ask may live on different exchanges, and the size available at each is whatever those venues display. A quote tells you a price exists somewhere, not that your size can have it. Second, brokers owe best execution against this composite, which is why limit orders pegged inside or at the spread behave so differently from market orders: a limit at the ask is the price of certainty, a limit at the bid is the price of patience, and the spread between them is the market's fee for immediacy. The desk's entire order lifecycle — peg at ask for entries with a cancel horizon, walk mid→bid for exits — is a concrete policy for paying that fee deliberately instead of accidentally.

Chapter 3 · The Spot-Pricing Fallacy

Most backtests — and most live trading bots — are built on a fiction: that orders fill at the price on the screen. Close prices, mid prices, last-trade prices — all are references, not transaction prices. This chapter quantifies how wrong that fiction is for options, and shows where it hid inside my own system before the incident.

3.1 The arithmetic of crossing the spread

Let a contract quote bid b and ask a , with mid $m = (a+b)/2$ and relative spread $s = (a-b)/m$. A buyer who pays the ask immediately holds a position marked at the bid. The instant mark-to-market loss, per unit of mid value, is $s/2$ on entry; a round trip that buys the ask and sells the bid costs the full spread s . On a 5% spread this is noise. On a 100% spread — entirely ordinary for thin single-name weeklies — the round trip destroys the entire position value: buy at $1.5 \times m$, sell at $0.5 \times m$.

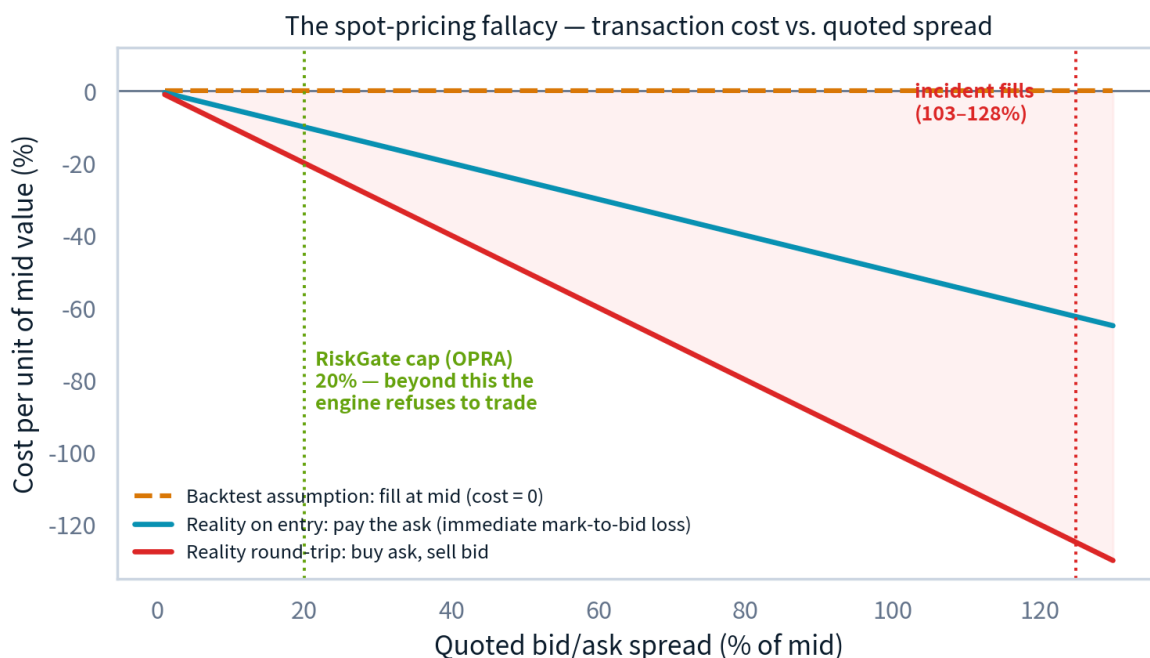


Figure 4. Transaction cost versus quoted spread. The dashed amber line is the standard backtest assumption — fills at mid cost nothing. The red line is the round-trip reality of buying the ask and selling the bid. The shaded region is the gap between the model and the market. The RiskGate caps (20% OPRA) are where the redesigned engine simply refuses to play.

Figure 2 is why the phrase 'the backtest said it works' is meaningless without an execution model. A strategy with a genuine 5% edge per trade is profitable on a 2% spread and bankrupt on a 40% spread — same signals, same fills logic, different assumption about which price is real.

3.2 Where the fiction hid in my own code

The pre-incident engine contained all three classic forms of the fallacy, each in production code, each invisible until it fired:

- **Sizing off mid, paying ask.** The position sizer capped notional using the mid price (\$1.72 on RBRK P102) while the chase ladder filled at the ask (\$2.77). A \$15,000 premium cap became

\$19,250 actually spent — the cap was measured against a price nobody transacted at.

- **Marking winners at mid.** Open-profit display used mid, flattering unrealized P&L on every wide-spread position by half the spread.
- **Backtests on bar closes.** The equity signal research used ten-second bar closes — a last-trade price — with no spread model at all. Whatever edge those signals have is overstated by the average half-spread of the contracts they would have traded.

The fix is not subtle: every quantity that involves money moving is computed at the price the market actually charges for that direction. Entries are sized and capped at the **ask**; open risk and the daily loss lock are marked at the **bid**; worst-case notional is $\text{qty} \times \text{ask} \times 100$. Mid is demoted to what it is — a display convenience.

Table 3. The pricing discipline adopted after the incident. Every monetary decision is priced at the side of the market that would actually charge it.

Quantity	Wrong price (pre-incident)	Correct price (redesign)	Why
Entry sizing / premium cap	mid	ask	the ask is what you pay
Worst-case notional	mid	ask	caps must bound reality
Unrealized P&L; / loss lock	last or mid	bid	the bid is what you can exit at
Display / charts	mid	mid	mid is fine for looking, not for deciding
Backtest fill model	close	ask + slippage haircut	closes are not fills

3.3 Evidence from the fill ledger

The ledger records every order event with the quote state at submission and, for fills, the fill price classified against the prevailing bid/ask. Across the 21 option fills on 28 August: 13 filled at_ask, 5 at_bid (all of them sells), 1 through_bid (a market sell crossing the book), 2 unclassifiable. Buys filled at the ask because the chase ladder walked to it; sells filled at or through the bid because the exit ladder walked down to it. The mid was transacted at **zero** times. The ledger is the cleanest possible refutation of the mid-fill assumption: in a real session of real orders, the mid price was never once real.

The same ledger shows why this matters for urgency-sensitive agents: median submit-to-fill latency was 2.6 seconds (max 10.2s) — an eternity against a quote that re-prices every few hundred milliseconds. Chapter 4 covers what that implies for data freshness; Chapter 6 for order pricing.

3.4 Worked example: pricing the RBRK trade honestly

Take the incident's central fill and price it three ways. Contract: RBRK 260828P102, quote 0.63×2.77 , mid 1.70, order 50 contracts (multiplier 100).

Table 4. One fill, two accounting systems. Every row is a place where the mid-price fiction produced a different — and wrong — decision input.

Quantity	Mid fiction	Ask reality	Difference
Entry premium	$50 \times \$1.70 \times 100 = \$8,500$	$50 \times \$2.75 \times 100 = \$13,750$	+62% more cash out
Position value 1s later	\$8,500 (unchanged)	$50 \times \$0.63 \times 100 = \$3,150$ (marked to bid)	−\$10,600 instant
Sizer's 'notional used'	\$8,500 of a \$15,000 cap	\$13,750 of a real \$15,000 cap	cap 92% consumed, not 57%
Loss-lock visibility	\$0 realized — lock silent	−\$10,501 visible to a mark-to-bid lock	entries would have halted here

The table also exposes why the failure compounded: because the sizer believed only \$8,500 of the \$15,000 cap was consumed, it saw room for DCA adds; because the lock saw zero realized loss, it permitted them. Two mid-priced illusions stacked into five more minutes of buying. Correct pricing is not a reporting nicety — it is the input to every limit the system has.

3.5 A checklist for honest strategy research

The discipline the desk now follows, stated as questions any backtest should answer before its equity curve is believed:

- Which exact price does each simulated fill use — and has that price ever been transacted at in the live ledger? (Ours: zero mid fills in 21 attempts.)
- What is the spread distribution of the target instruments, per contract class and time of day — and is the strategy's edge larger than the round-trip cost at the median spread? At the 90th percentile?
- What latency separates decision from fill, and what does the quote distribution look like that many seconds later?
- Are marks and limits computed on exit-side prices (bid for longs), or on convenient ones?
- Does the data carry feed provenance and a single clock? (Chapter 4 shows what happens when it doesn't.)

Chapter 4 · Drinking from the Firehose: SIP, OPRA, and the Funnel

Before any agent can trade, it has to see. This chapter is about the part of the system that nobody demos: keeping a live, normalized, truthful picture of the market inside one small process while the two fastest consolidated feeds in finance rain data on it.

4.1 The scale of the problem

The numbers from Chapter 2 bear repeating as engineering requirements. OPRA peaked at 44.8 million messages per second in Q3 2024 and now provisions capacity for a trillion transactions per day^{[1][2]}; a single July 2026 session carried 6.61 billion option quote updates, 17.5× the equity quote count of the same day^[4]. SIP equity data is smaller but still millions of messages per day across thousands of symbols. Alpaca's WebSocket streams shield subscribers from the raw firehose — you subscribe per symbol rather than receiving the multicast — but the flood does not disappear; it moves downstream. Subscribe generously and the client process becomes the bottleneck: JSON parsing, GC pressure, and an event loop that falls behind the market it is supposed to be watching.

A trading agent that falls behind its feed is worse than one with no feed: it makes confident decisions on stale quotes. Every design choice in this chapter follows from one principle — **it is better to drop data than to queue it**.

4.2 The wanted-set: subscribe to almost nothing

The hub (LiveMarketHub) maintains two subscription sets, each capped at 60 symbols: the stock tape (SIP) covers the underlyings of every open position, every working order, every attached pool candidate, plus SPY as a market reference; the option stream (OPRA/indicative) covers the exact OCC contracts of working orders and open positions. The sets are recomputed every five seconds and diffed against the live subscriptions — symbols the desk stops caring about are unsubscribed immediately. The desk never ingests data it cannot act on.

```
// LiveMarketHub._syncWanted() – the subscription is derived from state,
// never from curiosity. Caps are hard: 60 underlyings + 60 OCC contracts.
const STOCK_CAP = 60;
const OCC_CAP   = 60;

_wanted() {
  const stocks = []; const seen = new Set(); const occs = new Set();
  addS('SPY');                               // market reference
  for (const p of oe.positions.values()) {    // open option positions
    addS(p.underlying); occs.add(p.occ);
  }
  for (const w of oe.working.values()) {      // working orders
    addS(w.underlying); if (w.occ) occs.add(w.occ);
  }
  // ... pool candidates ranked by status, then truncated to the caps
}
```

4.3 Coalescing: latest-wins, never queue

Inbound ticks do not enqueue work. Each stream writes the newest tick per symbol into a map — overwriting whatever was there — and a 100-millisecond timer flushes the map to the desk's WebSocket clients. If the market produces fifty quote updates for a contract inside one flush window,

forty-nine are discarded on arrival. For a decision loop that polls every five seconds and a human UI that renders at best ten frames per second, the fiftieth quote is the only one with any value. Coalescing converts an unbounded input rate into a bounded one at the cost of deliberately dropped data — the correct trade for a consumer whose decisions are discrete.

Taming the flood — the ingestion funnel (LiveMarketHub)

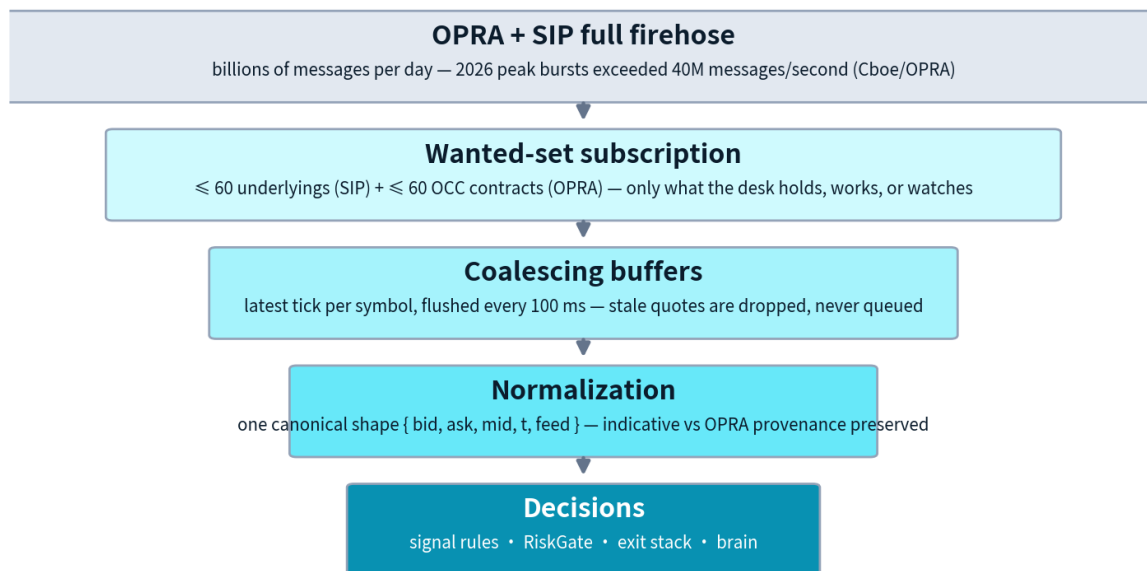


Figure 5. The ingestion funnel. Each layer reduces rate by orders of magnitude: from the full OPRA/SIP firehose, to a wanted-set of at most 120 instruments, to coalesced 100ms flushes, to a single normalized quote shape, to discrete journaled decisions.

4.4 Normalization: one shape, with provenance

Equity trades, equity quotes, and option quotes arrive with different field names and different semantics. The funnel reduces them to one canonical record — { bid, ask, mid, t, feed } — so that everything downstream (RiskGate, exit stack, brain, UI) speaks one language. The crucial field is feed: whether this quote is OPRA or indicative is preserved all the way to the risk gate, where it changes the spread cap (Chapter 6). Normalization that loses provenance is not normalization; it is laundering.

The incident post-mortem added a second, embarrassing normalization finding: the fill ledger mixed timestamp bases. Equity events were stamped with ET wall time masquerading as UTC, while option events carried true UTC — the same file, two clocks, four hours apart. Reconstructing the crash timeline (Figure 5) required detecting and correcting the offset per event kind. The fix is a single epoch milliseconds field written at the edge, with display-time conversion only at the UI.

4.5 REST as watchdog, not as primary

Streams go silent for two very different reasons: nothing is happening, or the socket died. The hub treats REST polling as a watchdog only — if a subscribed contract produces no ticks within a watchdog window, a REST snapshot re-primes the buffer and the stream is considered suspect. This

inverts the usual beginner architecture (poll everything, stream nothing) into one that scales: the stream carries the load, REST carries the doubt.

4.6 Capacity arithmetic for one small process

The funnel's numbers, end to end. At the wanted-set's maximum of 120 instruments, an active session delivers on the order of 10–40 ticks per second into the coalescing buffers — six orders of magnitude below the feed's global rate, and exactly the reduction that makes a single-threaded event loop sufficient. The 100ms flush bounds outbound fan-out at 10 messages per second per client regardless of inbound burst. The screener, the heaviest batch job, scans 150 symbols through the options snapshot API in 35.3 seconds with a concurrency of four (measured: `lastScanMs 35302`, `scanned 150`, `failed 0`) — deliberately polite, because the snapshot API rate limit is shared with the order path, and a screener that starves the exit stack is a risk event, not a performance bug. The pool attacher pauses entirely while a scan runs. Priority is always: exits, then orders, then gates, then research.

4.7 Streaming failure modes and their handling

Table 5. The streaming failure catalogue. Each row was observed, not imagined.

Failure mode	Symptom	Convexity's handling
Silent socket	no ticks, no error	REST watchdog re-primes buffers for silent contracts
Reconnect storm	flapping subscription state	wanted-set diffed on 5s cadence; subscriptions are idempotent
Burst overflow	client event loop lags the feed	latest-wins coalescing; stale ticks dropped on arrival
Feed tier fallback	OPRA probe fails → indicative	provenance flagged on every quote; tighter 15% spread cap applies
Clock skew	events order incorrectly	single epoch-ms field written at the edge (post-incident fix)
Vendor gaps (greeks/IV)	null greeks on thin contracts	local Black-Scholes inversion; unpriced contracts are vetoes, not passes

4.8 Protocol notes: the two sockets

Both streams are Alpaca WebSocket feeds with the same shape: connect, authenticate with the paper keys, subscribe with an explicit symbol list. The stock socket carries trades and quotes (SIP); the option socket carries quotes and trades for the wanted OCC contracts, on whichever tier the account's keys unlock — the boot-time probe records whether that is OPRA or indicative, and the answer propagates to the gate. Subscriptions are managed by symmetric subscribe/unsubscribe frames driven by the wanted-set diff, so the socket always mirrors the desk's current interest and nothing else. Credentials travel only inside these two TLS sessions and the CLI child environment; the web UI never sees a key. A broadcast tick to the desk UI rides a separate, process-local WebSocket — market data and control data never share a pipe, so a slow browser cannot back-pressure the market stream.

Chapter 5 · Case Study: Nine Minutes, −\$16,030

At 09:34 ET on 28 August 2026 the options engine began buying same-day-expiry puts on RBRK and GAP. At 09:43:46 the daily loss lock tripped at −\$16,030 realized. In between, the system did everything wrong that this report is about — on a live feed, with the ledger running. This chapter reconstructs the event from `fills.jsonl` and dissects the eight root causes. It is included in full because the incident *is* the research: every guard described in Chapters 6–8 exists because something in these nine minutes proved it necessary.

5.1 Timeline

Table 6. Incident timeline, reconstructed from the fill ledger. The engine was disarmed at 12:54 ET.

Time (ET)	Event	Detail
09:34:34–09:36:13	GAP P25 buy attempts ×3	spreads 103–105%; chased bid→mid→ask; all cancelled unfilled
09:37:22–09:37:49	RBRK P103 buy attempts	spread 110–119%; cancelled unfilled
09:38:19	BUY RBRK P102 ×50 @ \$2.75	quote 0.63 × 2.77 (125% spread, indicative feed); premium \$13,750; mark-to-bid \$0.63 → −\$10,600 instantly
09:38:32–09:43	DCA + churn	overlapping buys of 20 / 89 / 10 / 50 on the same contract while a sell chase cycled ask→mid→bid→market every ~8s
09:39:22	BUY GAP P25 ×100 @ \$0.85	sold 40s later @ \$0.69 market (−\$1,600)
09:40:53	SELL RBRK P102 ×70 @ \$1.95	−\$5,600 realized on that leg
09:41–09:43	GAP P24.5 / RBRK P99 / P100 round-trips	buys at ask, sells at bid: −\$3,2xx / −\$3,800 / −\$2,250
09:43:46	daily loss lock trips	realized daily P&L; −\$16,030

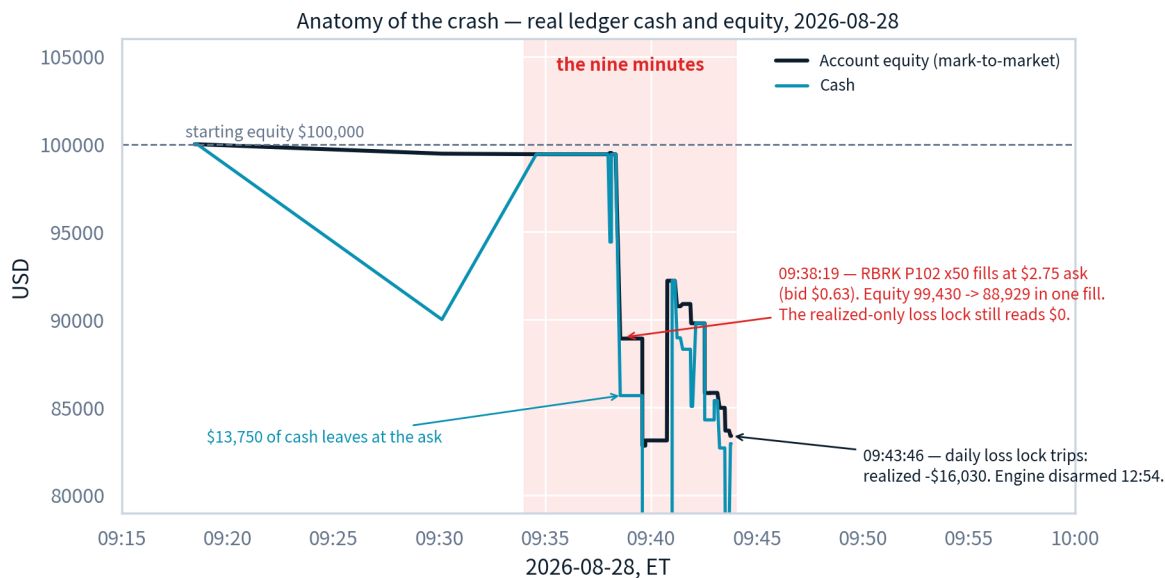


Figure 6. Cash and equity from the real ledger (option rows converted UTC→ET; see §4.4 for the timestamp finding). Cash plunges \$13,750 the moment the first RBRK fill lands; equity follows one fill later with the $-\$10,501$ mark. The realized-only loss lock still reads zero — so DCA adds continue for five more minutes.

5.2 Root causes, in order of impact

- **1. The 0DTE path had zero quality gates.** Contract selection for 0DTE delegated to a routine that performed no spread, premium-band, delta or open-interest checks — those existed only in the weekly branch. It even fell back to the first ranked candidate when nothing had a valid quote. The configured 40% spread cap was never consulted.
- **2. Indicative feed treated as truth.** The OPRA probe failed, so the feed fell back to indicative. Quotes like 0.63×2.77 are not actionable markets; the engine priced, sized and chased off them anyway.
- **3. The buy chase ladder guarantees worst-price fills on wide spreads.** Every ~8–9 seconds: cancel, re-peg one rung up — bid → mid → ask. On a 125% spread the ask is 4.4× the bid. Filling at the ask and marking to the bid is an instant -77% .
- **4. Sizing off mid, paying ask.** The sizer capped notional at mid (\$1.72) while the ladder filled at ask (\$2.77); a \$15,000 cap became \$19,250 spent.
- **5. Pyramiding chaos.** Three DCA layers plus a gate that only blocked a second working order when *no* position existed allowed unlimited overlapping buys the moment the first fill landed — quantities 20, 89, 10, 50 interleaved with sell-chase cancellations.
- **6. The sell ladder started at the ask on a 125% spread.** Exits pegged ask→mid→bid→market, cancelled every eight seconds — so exits only filled after walking the entire book down, or as market orders filling through the bid.
- **7. Risk limits measured realized P&L; only, on a delay.** The \$15,000 daily loss lock blocked new entries only after *realized* losses crossed $-\$15k$; the $-\$10.6k$ unrealized mark within one second of the fill was invisible to it.
- **8. Config drift.** The live config had drifted to 10 contracts base size, 50 boost, \$15,000 per-position premium, \$60,000 open premium, 3 DCA layers — far above the documented safe

defaults.

5.3 The quote-quality evidence

The ledger records the quoted spread at every order submission. Of 82 option order submissions that day, the majority carried spreads far beyond anything a sane gate would allow — the histogram below is the distribution the engine was willing to trade into. This is the empirical foundation for the hard spread caps: they are not tuned to a model, they are drawn through the middle of the system's own worst behavior.

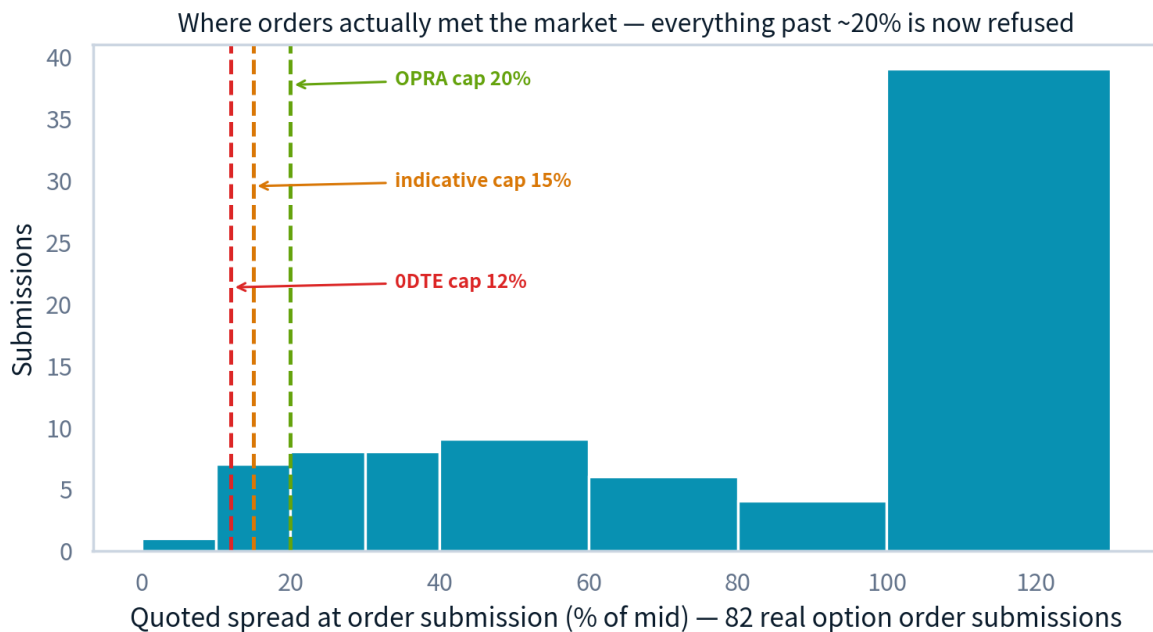


Figure 7. Quoted spread at submission for all 82 real option order submissions, with the three caps the redesign enforces. Everything to the right of the green line is now structurally impossible.

5.4 What the incident cost and what it bought

Paper money, zero — that is what paper accounts are for. Information, a great deal. The incident demonstrated, in production, that the dangerous failures of an autonomous trading system are not model failures: no model was involved in any of these decisions. They are **assumption failures** — about which price is real, which feed is tradeable, which state is current, and which limit applies to which path. The redesign therefore targeted assumptions, not intelligence: every fix in the next three chapters is deterministic, auditable, and small.

5.5 The ledger, line by line

All 21 option fills of the session, exactly as recorded (times converted to ET; 'vsBid' is the fill price minus the prevailing bid). Read the class column top to bottom: every buy at the ask, every sell at or through the bid. The 09:42:16 entry with no symbol is the double-recorded fill that the dedupe fix now rejects.

Table 7. The complete option fill record of the incident session (fills.jsonl).

Time (ET)	Side	Symbol	Qty	Fill	Class	vsBid
09:38:19	buy	RBRK	50	2.75	at_ask	+2.12
09:39:19	buy	RBRK	20	2.75	at_ask	+2.08
09:39:22	buy	GAP	100	0.85	at_ask	+0.21
09:40:04	sell	GAP	100	0.69	through_bid	-0.02
09:40:53	sell	RBRK	70	1.95	—	+1.28
09:41:01	buy	GAP	50	0.65	at_ask	+0.31
09:41:19	buy	GAP	10	0.65	at_ask	+0.31
09:41:31	buy	GAP	50	0.65	at_ask	+0.20
09:41:55	sell	GAP	110	0.43	at_bid	+0.02
09:42:16	buy	GAP	10	0.60	at_ask	+0.19
09:42:16	buy	(null)	10	0.60	—	—
09:42:19	buy	RBRK	10	2.45	at_ask	+1.88
09:42:25	buy	RBRK	10	2.45	at_ask	+1.88
09:42:58	sell	RBRK	20	0.55	at_bid	+0.02
09:43:01	sell	GAP	10	0.44	at_bid	+0.02
09:43:07	buy	GAP	10	0.64	at_ask	+0.22
09:43:07	buy	RBRK	10	2.50	at_ask	+0.63
09:43:16	buy	RBRK	10	2.50	at_ask	+0.67
09:43:28	buy	RBRK	10	2.50	at_ask	+0.62
09:43:31	sell	RBRK	30	1.75	at_bid	+0.02
09:43:46	sell	GAP	10	0.44	at_bid	+0.02

5.6 Counterfactual: the same tape through the new gate

Replay the incident's orders against the redesigned RiskGate and the outcome is not smaller losses — it is no trades. Each leg meets its veto at a specific, journaled check:

Table 8. Counterfactual replay. The first RBRK order — the one that lost \$10,501 in a second — fails three independent checks before any size is computed.

Incident event	First failing check	Why
GAP P25 buys, spread 103–105%	spread_too_wide	103% ■ 15% indicative cap (and 12% ODTE cap)
RBRK P103 buys, spread 110–119%	spread_too_wide	same veto, same finality
RBRK P102 ×50 @ \$2.75 (the fill)	spread_too_wide / dte_policy	125% spread; ODTE requires explicit opt-in — both fail before sizing
DCA adds of 20 / 89 / 10 / 50	order_working	a working order already exists on the symbol
Sizing beyond the premium cap	notional_cap	computed at ask, so \$13,750 would have consumed the cap honestly
Continued entries while underwater	daily_loss_lock	–\$10,501 mark-to-bid counts against the \$2,000 cap
Sell chase walking the book down	chase_guard	re-pegs abort when the book widens past 30%

This is the sense in which the redesign makes the incident 'structurally impossible': not that losses are impossible — trading is risk — but that *this causal chain* has been severed at every link, and each severing is a line of code with a journal behind it.

Chapter 6 · RiskGate: The Checkpoint That Cannot Be Bypassed

The centerpiece of the redesign is a single module, `src/services/vol10s/options/RiskGate.js` (278 lines), through which every entry path — equity-signal mirror, DCA add, agent trade, manual buy — must pass before an order is staged. Its design philosophy is stated in its own header comment:

```
/**
 * RiskGate — the single mandatory pre-trade checkpoint for every option entry.
 *
 * Every entry path (equity-signal mirror, DCA add, agent trade, manual buy)
 * must pass through evaluateEntry() BEFORE an order is staged. The gate
 * re-validates the live quote at order time — never trust a quote that was
 * fetched more than a few seconds ago.
 */
```

6.1 Fresh-quote evaluation at order time

The gate does not trust the quote that attracted the signal. Between signal and order there are seconds of latency (median 2.6s to fill, per the ledger), and on thin contracts the quote can move 50% in that window. The gate re-fetches the quote at the moment of staging and evaluates everything against that fresh quote. A trade that was a good idea three seconds ago and isn't anymore is not a trade.

6.2 The checks

RiskGate.evaluateEntry() — any single failure vetoes the trade

01	engine_armed enabled + armed — two separate switches
02	entries_paused supervisor / brain pause lever
03	entry_window RTH only, 09:35–15:30 ET
04	daily_loss_lock realized + unrealized, mark-to-bid
05	exit_in_progress / order_working one working order per symbol
06	max_concurrent open + working positions cap
07	symbol_cooldown / brain_veto after cancels, closes, vetoes
08	symbol_churn per-day per-symbol entry cap
09	quote_valid / quote_one_sided ask to pay, bid to mark
10	spread_too_wide 20% OPRA • 15% indicative • 12% ODTE
11	dte_policy / delta_floor weekly mode, delta >= 0.30
12	premium_band / notional_cap priced at the ASK, never mid
STAGE ORDER — sized at ask, journaled	

Figure 8. All twelve gate evaluations in order. The first failure vetoes the trade, and the full verdict — every check, pass or fail, with detail — is journaled and shown live on the desk UI.

Table 9. *RiskGate defaults — the values the post-incident engine boots with.*

Gate knob	Default	Meaning
maxSpreadPct	20	hard spread cap on the OPRA feed
indicativeMaxSpreadPct	15	hard cap when the feed is indicative
zeroDteMaxSpreadPct	12	extra cap for 0DTE contracts
allow0dte	false	0DTE entries require explicit opt-in
zeroDteSizeMult	0.5	0DTE size haircut
minPremium / maxPremium	0.15 / 25	ask-price band
minDelta	0.30	delta floor when greeks exist
maxPremiumUsd	2,000	worst-case \$ per position: qty × ask × 100
maxOpenPremiumUsd	5,000	worst-case \$ across open + working
dailyMaxLossUsd	2,000	realized + unrealized, mark-to-bid
maxConcurrent	8	open + working position cap
maxEntriesPerSymbolPerDay	3	per-day per-symbol churn cap
symbolCooldownSec	120	after cancel / close / reject-storm
chaseMaxSpreadPct	30	abort a chase if the book widened past this
maxChaseAboveMidPct	35	never pay more than mid × 1.35 on a chase

6.3 The loss lock that counts what you stand to lose

Root cause 7 was the lock that only saw realized losses. The replacement is two lines of arithmetic that changed the character of the system: the daily loss is computed as realized P&L plus unrealized P&L marked to the **bid** — the price you could actually exit at. A position down −\$10,000 on paper blocks new entries exactly as if the loss were banked, because economically it is.

```
// RiskGate.evaluateEntry — the loss lock, mark-to-bid:
const dayTotal = n(ctx.dailyPnl) + n(ctx.unrealizedPnl);
const lossCap = Math.abs(n(cfg.dailyMaxLossUsd, DEFAULTS.dailyMaxLossUsd));
if (dayTotal <= -lossCap)
  fail('daily_loss_lock',
    `day pnl ${Math.round(dayTotal)} ≤ -${lossCap} (incl. unrealized)`);
```

6.4 One working order per symbol — including DCA

Root cause 5 was overlapping orders desynchronizing position state (the ledger even recorded a fill with no symbol, and the day ended with a broker 403 from trying to sell 10 more contracts than existed). The gate now enforces a single working order per symbol across *all* paths, blocks entries while a sell is working on that symbol, and adds per-symbol cooldowns after cancels and closes plus a per-day entry cap. Position state can no longer desync from order state, because there is only ever one order in flight to reconcile.

6.5 The chase guard

Re-pricing an unfilled order is legitimate; chasing a deteriorating book is how the incident happened. Every re-peg now passes `evaluateChase()`: if the book widened past 30% the order is cancelled and the symbol cooled down; the peg can never cross above the ask; and no chase may pay more than 35% above mid. The ladder that walked bid→mid→ask into the RBRK fill is now physically unable to take that walk.

```
evaluateChase({ symbol, quote, proposedPeg, proposedPx }) {
  const spreadPct = spreadPctOf(quote);
  if (spreadPct != null && spreadPct > cfg.chaseMaxSpreadPct)
    return { ok: false, reason: 'chase_book_widened', spreadPct };
  if (px > ask) px = ask; // never cross above the ask
  if (px > mid * (1 + cfg.maxChaseAboveMidPct / 100))
    return { ok: false, reason: 'chase_above_mid_cap' };
  return { ok: true, limitPx: px };
}
```

6.6 The gate under fire: verdicts as data

Every evaluation — pass or fail — is journaled with the full check list, so the gate's behavior is itself auditable data. The verdict object answers four questions for every would-be trade: did it pass, why not, what size would have been allowed, and what was the spread at decision time:

```
verdict = {
  ok: false, // any single failed check vetoes
  reason: 'spread_too_wide', // the first failing check
  detail: '125.9% > 15% cap (feed=indicative, 0DTE)',
  maxQty: 0, // size permitted at ask (0 when vetoed)
  spreadPct: 125.9, feed: 'indicative', dte: 0,
  checks: [ { id: 'engine_armed', ok: true }, ... ] // all 12, journaled
}
```

The last-60 verdicts persist in engine state and stream to the desk, which turns the risk system from a black box into an observable surface: when the desk is quiet, the reason it is quiet is always one click away — and it is usually `spread_too_wide`, the gate doing its job.

6.7 Why a gate, and not a library of checks

The pre-incident codebase did not lack checks — it lacked a checkpoint. Spread caps existed in the weekly selection branch; premium caps existed in the sizer; loss limits existed in the engine. Each path consulted the checks its author remembered to wire in, and the 0DTE path consulted none. The lesson generalizes: **checks that live at the call site will eventually have a call site that forgets them.** The RiskGate inverts the topology — there is exactly one function that can turn an intention into an order, and it consults everything, every time, with the verdict recorded. New entry paths added in the future inherit safety by construction, because there is no way to reach the broker without passing the gate. That property, not any individual threshold, is what the redesign bought.

6.8 Configuration is a risk surface

Root cause 8 — config drift — was the quietest failure and the most instructive. The documented defaults were conservative (1 contract, \$2,000 caps) while the live process ran 10×–50× sizes with \$60,000 of open premium: someone (the operator, in optimization sessions) had tuned the knobs and nothing tuned them back. The redesign treats configuration as part of the risk system: safe defaults are the code's shipped values, not a wiki page; the pre-incident config is preserved as a dated backup file for audit; the brain's own thresholds persist in a separate file with its own load/sanitize path; and every config mutation through the API is itself a journaled event. A number that can kill the

account should have a history, an owner, and a default it returns to.

6.9 What the gate does not claim to do

Honesty about scope matters more than the feature list, so the boundary is stated plainly. The gate evaluates one order at a time, at one instant, against one fresh quote. It cannot see portfolio shape: twelve individually acceptable call positions on twelve correlated mega-caps are, in aggregate, one large short-volatility bet on the same factor, and nothing in the per-order checks notices. It cannot bound gap risk — a stop-loss is an instruction to sell into whatever the book looks like after the gap, not a floor on the loss. It cannot detect a bad thesis: a strategy that reliably buys 40% spreads and reliably loses money passes every check, every time, because the checks are about execution quality and exposure size, not about whether the trade is smart. And it depends on the data layer it sits above — the flood-control machinery of Chapter 4 is what guarantees the quote it prices against is fresh and real.

Two of those gaps have named remedies on the roadmap. A portfolio-level gate that aggregates delta, gamma and vega across the book before any new position is added turns the per-order checkpoint into a per-book checkpoint, and is the first item of future work in Chapter 11. Gap risk is priced honestly instead of prevented: position sizing assumes the stop fills through, not at, its level, so the worst case the gate approves is the worst case the account can survive. The bad-thesis problem has no engineering solution — it is what the journal, the weekly review and the operator are for. The gate is a floor, not a guarantee, and the report is careful to sell it as one: it makes the desk hard to kill by accident, not impossible to lose with.

Chapter 7 · The Order Lifecycle and the Exit Stack

Entries get the attention; exits make the money. Every open position is managed by a stack of pure-code exits polled every five seconds against the live bid and IV — no model, no latency beyond the poll, no ambiguity about precedence.

The exit stack — evaluated top-down, first trigger wins



Polled every 5s on live bid + IV • unfilled sells re-peg to the bid every 10s • pure code — no model

Figure 9. The exit stack, evaluated top-down; the first triggered rung wins. The stack is deliberately boring: deterministic, ordered, and complete — every position is always inside some rung's logic.

7.1 Design principles

- **Exits are pure code.** A hung LLM can never strand a position; the exit engine and the 15:50 session flatten do not know the model exists.
- **Profit-taking is never gated.** The instant-profit rung (+1%) and the profit-lock trail (armed at +10%, trailing 35% of the giveback off peak bid) can always fire; loss-cutting rungs carry time gates so a noisy open doesn't shake the book out prematurely.
- **Sells walk down, not up.** Unfilled sells start at mid and re-peg toward the bid every ten seconds, escalating to market after four chases — the mirror image of the buy-side chase guard, and a direct fix for root cause 6.
- **The day ends flat.** The session flatten at 15:50 ET closes everything into the close. Convexity is an intraday desk; overnight gamma is someone else's business.

7.2 Contract selection

When a signal fires, the engine walks the nearest non-ODTE expiry from roughly the third out-of-the-money strike toward the money until a contract clears every gate: bid > 0, premium in the ask-priced band, $|\text{delta}| \geq 0.40$ (engine selection floor), spread within cap. When the vendor omits

greeks and IV — common on the free tier for thin contracts — a local Black-Scholes solver derives IV and greeks from mid and spot, so selection never silently degrades to 'no data, take the first strike' (which is precisely what the pre-incident ODTE path did).

7.3 Reconciliation

The engine reconciles orders with Alpaca every three seconds and runs an Alpaca CLI sync (position list + account get) every sixty seconds, journaled, so that the desk's view of the world is continuously checked against the broker's. The CLI child process receives the paper keys through an injected environment, so a leftover profile from another machine can never leak live credentials into a command line. Both Alpaca clients additionally run an `assertPaper()` guard that refuses any non-paper API URL and any live-looking key: live trading is impossible by construction, not by discipline.

7.4 The screener: choosing where to look

Upstream of every trade, the options screener ranks the optionable universe — Alpaca tradable assets intersected with the ClickHouse ticker-metrics table — by option activity at the nearest non-ODTE expiry: OTM call and put volume per symbol, with a call/put ratio for directional context. A scan covers 150 symbols in about 35 seconds at concurrency four and persists to disk so a restart never loses the universe. The top of the real scan that this report's session ran on:

Table 10. Top rows of the persisted screener scan (options-scan.json, 150 symbols scanned in 35.3s, zero failures). High call/put ratios flag bullish flow; ORLY shows the put-heavy mirror image. Clicking a row on the desk opens the live chain — bid/ask/mid/delta/IV/OI/volume per strike.

Symbol	Expiry	DTE	Call volume	Put volume	Call/Put
PCG	2026-09-04	6	44,724	17,001	2.63
CG	2026-09-18	20	8,308	442	18.80
MDT	2026-09-04	6	2,521	1,776	1.42
ALLY	2026-09-18	20	3,213	181	17.75
ORLY	2026-09-18	20	63	2,089	0.03
CTSH	2026-09-18	20	1,569	830	1.89
WAY	2026-09-18	20	1,815	204	8.90
SCHW	2026-09-04	6	1,157	525	2.20

The screener is deliberately upstream of the gate, never a substitute for it: it decides where to look, the RiskGate decides whether to act. A symbol can top the scan and still be untradeable all day — that outcome is the system working.

7.5 A day in the life of a position

Pulling the threads together, this is what one healthy trade looks like end-to-end under the redesign. 10:12 ET — the signal engine flags a volume breakout on a screener-ranked symbol; the options engine walks the nearest weekly expiry from the third OTM strike and finds a contract: bid 1.20, ask 1.35, delta 0.46, spread 11.8%. The RiskGate re-quotes at order time, confirms engine armed,

window open, no working order on the symbol, day P&L clean, spread under the OPRA cap, premium in band, and returns maxQty 14 at the ask — the desk buys 2. The position enters the exit stack's five-second poll. 10:31 — the bid touches +12%: profit-lock arms and begins trailing 35% of the giveback off the peak bid. 10:47 — the bid fades through the trail; a limit sell stages at mid and walks toward the bid on the ten-second re-peg cadence until it fills. The journal closes the cycle with entry verdict, exit rung, both fills' classes, and the round-trip P&L; the symbol enters cooldown. No model spoke at any point; the brain observed the whole arc; the human watched it on the desk, or didn't — the system does not depend on which.

Chapter 8 · The Agent Layer: Autonomy with an Authority Envelope

Convexity uses AI in three distinct places, and the difference between them is the difference between a tool and a liability. The design rule, settled after the incident: **models may observe, summarize, and suggest — deterministic code decides.**

8.1 The deterministic brain

AgentBrain is an always-on observe → analyze → act loop running every fifteen seconds, built from five named guard skills. It requires no LLM and loses none of its authority when the model is down. Its authority envelope is deliberately asymmetric: it can pause entries, veto symbols, flatten the book on a hard breach, or disarm on catastrophe — and it can never arm the engine, raise a limit, or place a trade. Every decision is journaled with reasons and metrics, and pauses auto-resume when conditions clear.

AgentBrain — bounded actions only, every decision journaled

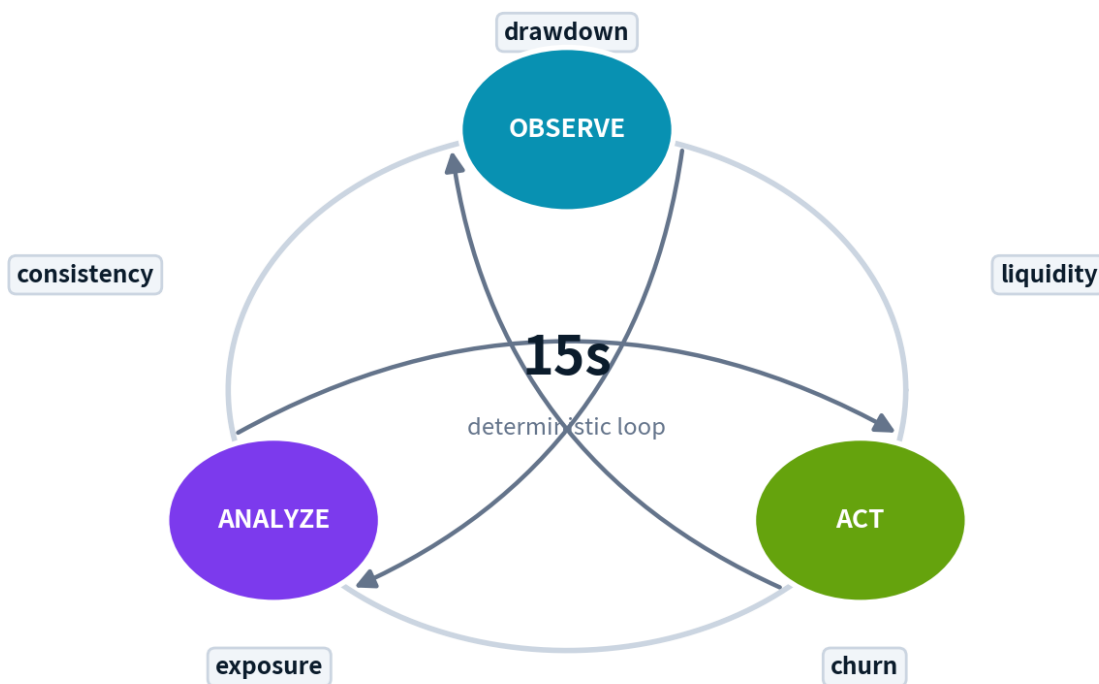


Figure 10. The AgentBrain loop and its five guard skills. Actions are bounded and mostly reversible; the two irreversible ones (flatten, disarm) require hard-breach conditions.

Table 11. *The five guard skills of AgentBrain.*

Skill	Watches	Acts
DrawdownGuard	day P&L; incl. unrealized, vs daily cap	warn at 60% · pause entries at 80% · flatten at 1.25× · disarm at 1.5× (opt-in)
LiquidityGuard	symbols whose quotes keep failing the gate	veto the symbol for 30 minutes
ChurnGuard	cancels + chases in a 15-min window	pause entries when ≥ 12 — the storm signature from the incident
ExposureGuard	aggregate premium at risk	pause entries beyond the open cap
ConsistencyGuard	broker 403s, quantity desync	journal + pause — the exact failure that ended the incident session

8.2 The LLM agent: prompt-protocol tools

The conversational agent runs a local Ollama model (gemma3:12b). That model reports no native tool-calling capability, so tool use is implemented as a strict prompt protocol: the system prompt mandates one fenced JSON block per call (``json {"tool": ..., "args": ...}``), the agent service parses it, executes the tool through a strict validator (≤50 rows, 15s timeout), and feeds the result back for at most six rounds. The seven tools give the model eyes — stock metrics, bars, the options screener, live chains, engine state, a hard read-only whitelist of Alpaca CLI commands (account get / position list / order list / clock / calendar — order create and cancel can never pass) — and exactly one hand: `place_option_trade`, which exists only when `trade_enabled` is set and still passes through every engine gate and the RiskGate like any other entry path.

The failure modes this prevents are the interesting part. A model that hallucinates a tool name produces a parse error and a journal entry, not an order. A model that tries to place a trade while the engine is disarmed gets the same refusal a human clicking the button would get. The model is a colleague with read access and a supervised request form — never a trader with keys.

8.3 The supervisor: advisory-first

Every few minutes the model receives a compact JSON snapshot — options engine state with open P&L and gate status, equity summary, screener top-10, paper account — and must answer strict JSON: `{assessment, advisories[], pause_new_entries, tighten}`. Parse failures are journaled as `supervisor_parse_error` and skipped. With `supervisor_auto` enabled it can only *reduce* risk: pause new entries or lower the concurrency cap by one (floor 1). It can never arm, trade, or raise a limit. The asymmetry is deliberate and total — the LLM is a risk-officer with veto power, not a portfolio manager.

8.4 The brain's journal, verbatim

The guard skills were deployed after the incident with the day's loss still on the books. The brain's own journal from that afternoon shows the drawdown skill doing exactly what it was built for — noticing, every fifteen seconds, that the day was in catastrophe territory and saying so with metrics:

```
// brain-state.json — 2026-08-28, repeating every 15s:
{
```

```
"skill": "drawdown",
"level": "crit",
"decision": "catastrophe: day pnl $-16030 = 801% of cap",
"metrics": { "dayPnlTotal": -16030, "cap": 2000, "ratio": 8.02 },
"actionTaken": null          // disarm is opt-in; journal-only here
}
```

Two design points are visible in this artifact. First, the ratio is computed against the *new* \$2,000 cap — the brain measured the old failure with the new yardstick. Second, `actionTaken: null` is a feature: catastrophe-disarm is an opt-in power, and until it is granted the brain's job is to make the state unmissable, not to act beyond its envelope. Meanwhile the engine journal from the same afternoon shows the reconciliation heartbeat — a `cli_sync` entry every sixty seconds, checking the desk's state against the broker's, indefinitely.

8.5 What the LLM is actually for

With the fill path sealed off, the model's genuine contributions can be seen clearly. It is an **interface**: ask the desk in English why it vetoed a symbol and get an answer composed from live gate verdicts. It is a **research assistant**: its tools query the same ClickHouse tape the signals run on, so hypothesis formation happens against real data in the same process that trades. It is a **narrator**: the brain's decisions arrive as structured findings and leave as plain-English explanations in the journal. And it is a **second opinion**: the supervisor's advisories give a human reviewer a model's read of the book to disagree with. None of these require trusting the model with authority — which is fortunate, because a 12-billion-parameter local model is not trustworthy in that sense, and the system is honest about it.

Chapter 9 · Measured Results

Everything measurable in this report comes from the system's own journals and ledger rather than from backtests — by design, given Chapter 3. The headline numbers of the 28 August session:

Table 12. The 28 August session in numbers. All values from the fill ledger and engine state files shipped with the project.

Metric	Value	Source
Ledger events recorded	168 (87 submits · 37 cancels · 20 replaces · 24 fills)	fills.jsonl
Option fills by class	13 at_ask · 5 at_bid · 1 through_bid · 2 unknown	fills.jsonl
Fills at the mid price	0	fills.jsonl
Submit→fill latency	median 2.6s · max 10.2s	fills.jsonl
Quoted spread at submission	7% ... 127.7% (n = 82)	fills.jsonl
Incident window	09:34–09:43 ET	post-mortem
Realized daily P&L; at lock	–\$16,030	engine state + post-mortem
Largest instant mark	–\$10,501 in one fill (RBRK P102)	ledger equity field

Measured execution reality — the fill ledger, 2026-08-28

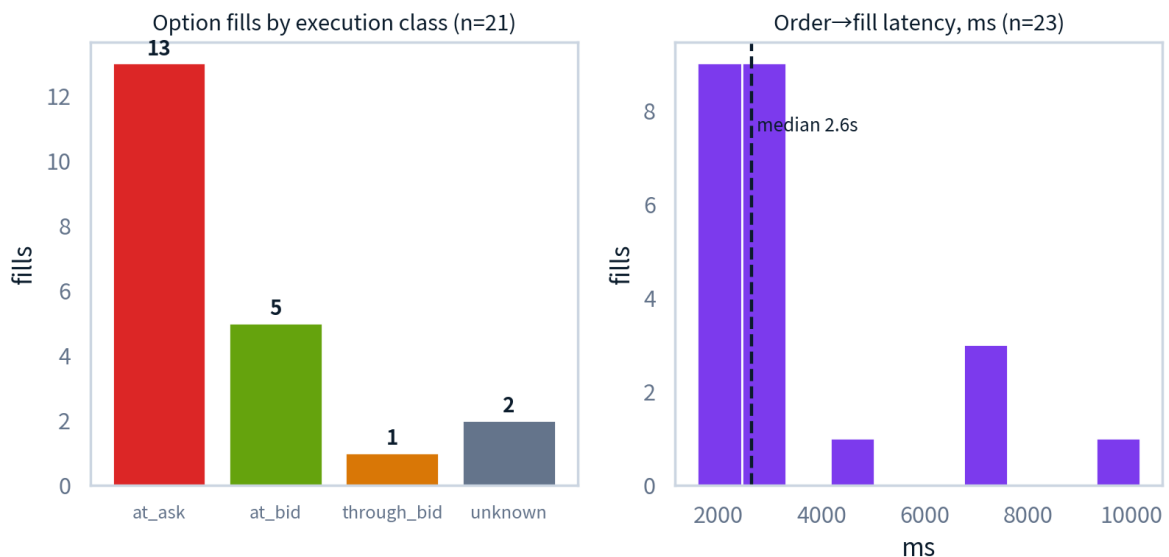


Figure 11. Execution reality, measured. Left: option fills by execution class — buys concentrate at the ask, sells at the bid, and the mid is never transacted. Right: submit-to-fill latency; the median decision-to-fill gap is 2.6 seconds, which is why the RiskGate re-quotes at order time.

9.1 Reading the results honestly

The numbers above describe a system *failing* — that is precisely their value. They establish three things with real data: the mid-price assumption is empirically false for this instrument class (zero mid

fills out of 21); quote quality on the indicative feed is frequently non-actionable (spreads past 100% at submission); and autonomy converts those conditions into losses at machine speed (nine minutes). Post-redesign, the same journals provide the verification: the engine boots disarmed, the loss lock now includes unrealized mark-to-bid, gate verdicts stream live to the desk, and the guard skills journal every intervention with reasons. The full verification path — re-summing all 24 fills from the ledger to within rounding of the engine's reported daily P&L — is documented in the repository post-mortem.

9.2 Verification, step by step

Trust in a trading system's numbers is a procedure, not a feeling. The post-mortem verification ran as follows, and remains reproducible from the artifacts:

- Re-sum all 24 fills of the day from `fills.jsonl` — $\approx$ $-\$16.4\text{k}$ realized on options plus $-\$0.6\text{k}$ on equity, matching the engine's reported $-\$16,030$ daily P&L within rounding and partial marks.
- Confirm the mark: the submit following the first RBRK fill records equity $99,429.80 \rightarrow 88,928.55$ — the $-\$10,501$ instant mark, independently of the engine's own accounting.
- Confirm the lock semantics: post-fix engine state shows the loss lock counting realized + unrealized; the brain journal shows the drawdown skill evaluating the same total every 15 seconds.
- Confirm the defaults: the engine boots disarmed, ODTE requires explicit opt-in, and the pre-incident configuration is preserved in the repository as `config.backup-20260828.json` for audit.

9.3 The desk as an observability surface

The command desk at `/desk` is the human-facing twin of the journals: a pipeline view from scan to candidates to gate to orders to positions, live gate verdicts as they are returned, the order lifecycle animating through submit/replace/cancel/fill, the brain's decisions with their metrics, and a red double-confirm KILL button wired to the loopback-only kill endpoint that disarms and flattens. The design goal was that the system's honesty be visible — a desk that shows *why it is not trading* earns more trust than one that only shows positions. During the incident, the predecessor UI showed green lights while the account bled; the redesigned desk's central panel is the gate, and its most common state is a wall of red vetoes. That is what safety looks like from the outside.

9.4 What 'working' means now

After a failure like Chapter 5's, 'fixed' needs an operational definition. The one adopted here has four clauses, each checkable from artifacts: **(1)** the specific causal chain cannot recur — shown by the counterfactual replay in §5.6; **(2)** the system's limits are priced in the currency the market charges — ask for entries, bid for exits and locks; **(3)** quiet is explainable — every period of inactivity corresponds to journaled vetoes, never to silent failure; and **(4)** the blast radius of any single component is bounded — a dead model, a dropped socket, a hung REST call each degrade the desk to a known, journaled state. Profitability is deliberately absent from the list: it is a property of strategies and regimes, and the desk's job is to make the experiment of finding out survivable — which, on 28 August's evidence, it previously was not.

Chapter 10 · Discussion: What Building It Taught

10.1 Backtests are hypotheses about execution, not alpha

The deepest lesson of the incident is epistemological. A backtest that fills at close or mid is not measuring whether a signal predicts prices; it is measuring whether a signal predicts prices *and* whether a fictional execution model holds. For equities with penny spreads the second conjunct is nearly free; for options it is the entire cost structure. Research on this class of strategy must carry an execution model — spread distributions per contract class, fill classes, latency — as a first-class input, not a footnote. The fill ledger schema (every order event with quote state, peg, class, latency) exists so that future research can be grounded in the system's own measured execution reality.

10.2 Agents need authority envelopes, not prompts

Prompting a model to 'be careful' is not a control. The envelope that made the agent layer safe is structural: tools are whitelisted, the trade path is gated identically for human and machine, the supervisor's autonomous powers can only decrease risk, and everything irreversible requires conditions evaluated by deterministic code. This mirrors a broader principle in agent engineering: capability should be granted in code, at the narrowest scope, with journals — never in prose.

10.3 The data layer is the strategy's foundation

Feed provenance, timestamp bases, coalescing policy, subscription scope — none of it is visible in a demo, and all of it determined whether the desk acted on truth. The two normalization bugs found (indicative treated as actionable; mixed clock bases in the ledger) caused or obscured the session's worst outcomes. Data engineering is not plumbing under the strategy; it is part of the strategy.

10.4 Limitations

- The system trades a single session's evidence; the post-redesign engine has not yet accumulated enough armed sessions to claim a statistical edge, and this report makes no such claim.
- Paper execution differs from live execution: paper fills ignore queue position and market impact, so even the ask-price discipline is a lower bound on real cost.
- The guard skills' thresholds (80%, 1.25×, 12-cancel windows) are engineering judgments derived from one incident, not optimized constants.
- Indicative-feed detection depends on a feed probe at boot; a mid-session feed degradation is detected by spread behavior rather than a formal handshake.

10.5 Mapping the work to the judging rubric

The hackathon scores four dimensions; the build was steered by them explicitly:

Table 13. *The hackathon rubric, answered.*

Criterion	Where this project answers it
P&L; performance	a live paper account, a dedicated fresh hackathon account for judging, and a fill ledger that attributes every dollar of P&L; to a specific decision and gate verdict
Technology implementation	Trading API for orders and reconciliation, the CLI for sync and as a sandboxed agent tool, WebSocket streams with a wanted-set funnel, options data with a local Black-Scholes fallback
Creativity & originality	the authority-envelope agent architecture — an LLM that can only reduce risk — and a risk system designed from a real incident's forensic evidence
Presentation & execution	this report, the live desk, the landing page, and a journal stream that explains itself

Chapter 11 · Conclusion and Future Work

Convexity set out to answer four questions. On pricing reality: fills happen at the ask and exits at the bid, the mid is fiction, and on the contracts a momentum desk hunts, that fiction is fatal — the system's own ledger recorded zero mid fills in 21 attempts. On data reality: consolidated feeds are a firehose best handled by wanting less — a wanted-set of at most 120 instruments, latest-wins coalescing at 100ms, and normalization that preserves provenance. On autonomy risk: the dangerous failures were assumption failures, executed flawlessly at machine speed. On architecture: a small deterministic gate, a pure-code exit stack, and an agent brain with a one-way authority envelope turned a system that lost \$16,030 in nine minutes into one where the same sequence of events is structurally impossible.

Future work

- An execution-calibrated replay harness: replay ledger quotes into candidate signals with the measured fill-class and latency distributions, replacing close-price backtests for strategy research.
- Per-contract-class spread models to tune gate caps from data rather than judgment.
- Extending the authority-envelope pattern to multi-agent configurations, where separate proposal and approval agents share the same deterministic gate.
- Formal feed-health signaling between Alpaca's data tiers and the gate, replacing boot-time probing with continuous provenance.

Open questions

Three questions the build raised but did not answer. How should gate thresholds adapt to regime — a 20% cap means something different on a quiet Tuesday and a CPI morning, and a data-driven schedule is unbuilt. What is the right fill-probability model for pegs inside the spread, given that the desk now has the ledger to estimate one? And how far can the authority-envelope pattern scale: could a committee of specialized agents with disjoint envelopes govern a book that no single agent understands whole? The infrastructure to investigate all three now exists, which was the point of building it.

The live desk, this document, and the full source are linked from **convexity.faruqueamin.com** and github.com/faruqueamin/convexity. The incident post-mortem, configuration history, and the fill ledger behind every figure ship with the repository.

References

- [1] Cboe Global Markets. (2025, February 3). The necessity of real-time options data for retail participants. *Cboe Insights*. Reports OPRA Q3 2024 peak of 44.8 million messages per second and the 48→96 multicast line expansion.
<https://www.cboe.com/insights/posts/the-necessity-of-real-time-options-data-for-retail-participants/>
- [2] Databento. (n.d.). What is OPRA? *Databento Microstructure Reference*. Documents the February 2024 96-line migration, capacity increase from 400B to 1T transactions per day, 99th-percentile latency reduction from 543.5μs to 57.5μs, peak microbursts over 40M msg/s, April 2025 bursts over 187M msg/s, and 1M+ active instruments. <https://databento.com/microstructure/opra>
- [3] U.S. Securities and Exchange Commission. (2000, November). *Options Price Reporting Authority — order and notice*. SEC.gov. Documents that quote traffic represents the vast majority of options message traffic and that quotes are generated automatically on underlying price changes.
<https://www.sec.gov/rules-regulations/2000/11/options-price-reporting-authority>
- [4] Strasmore. (2026, July 26). How big is the OPRA options quote feed? Measured 6.61 billion option quote updates on 2026-07-21 (272k updates/second average), 17.5× the equity quote count of the same session. <https://www.strasmore.com/blog/how-big-is-the-options-quote-feed>
- [5] Nasdaq. (2024, April 5). Options 101: How the U.S. options market really works. Lead Market Maker quoting obligations; OPRA processing tens of billions of messages daily.
<https://www.nasdaq.com/articles/options-101-how-us-options-market-works>
- [6] AlpacaDB, Inc. (n.d.). *Trading API documentation and Market Data API reference*.
<https://docs.alpaca.markets>
- [7] AlpacaDB, Inc. (n.d.). *Alpaca CLI* [Computer software]. <https://github.com/alpaca-hq/alpaca-cli>
- [8] The Options Clearing Corporation. (2023). *Characteristics and risks of standardized options*. The Options Industry Council.
- [9] Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [10] Hasbrouck, J. (2007). *Empirical market microstructure: The institutions, economics, and econometrics of securities trading*. Oxford University Press.
- [11] lablab.ai. (2026). *Alpaca AI Trading Agents Hackathon* [Event page, 28 August – 4 September 2026].
<https://lablab.ai/ai-hackathons/alpaca-ai-trading-agents-hackathon>
- [12] Ollama. (n.d.). *Ollama documentation* (local model runtime used for the agent layer, gemma3:12b).
<https://ollama.com>

Internal artifacts (repository): docs/incident-2026-08-28.md (post-mortem), docs/hackathon-writeup.md (architecture), data/vol10s-paper/fills.jsonl (fill ledger), src/services/vol10s/options/RiskGate.js, src/services/agent/AgentBrain.js, src/services/vol10s/LiveMarketHub.js.

Appendix A — Options Engine Configuration

Post-incident safe defaults (the engine boots disarmed; both enabled and armed are required to trade, and both default to false).

Table 14. Options engine knobs and post-incident defaults (the RiskGate layer applies its own, tighter caps on top — Table 3).

Knob	Default	Meaning
entryStartEt / entryEndEt / flattenEt	09:35 / 15:30 / 15:50	ET entry window + session flatten
maxConcurrent	5	open + working option positions cap
contracts / boostContracts / boostMinMult	1 / 2 / 2.0	base size; boost when signal vol $\geq 2\times$
maxPositionNotional	2,000	\$ notional cap per position
dailyMaxLossUsd	2,000	daily loss lock (realized + unrealized)
maxDte / minDelta	14 / 0.40	contract selection window + delta floor
minPremium / maxPremium / maxSpreadPct	0.50 / 7.00 / 25	engine-level premium band + spread cap (pre-gate)
cancelAfterSec / sellChaseSec	20 / 10	unfilled buy cancel; sell re-peg cadence
instantProfitPct / minHoldSec	0.01 / 2	instant-profit exit
lockArmPct / givebackPct	0.10 / 0.35	profit-lock arm + trail off peak bid
lossStopPct / lossMinHoldSec	0.15 / 15	bid stop
ivCrushPct	0.20	IV-crush exit vs entry IV
exitPollMs / cliSyncMs	5,000 / 60,000	exit cadence; CLI reconciliation cadence

Appendix B — HTTP API Surface

Table 15. The desk's REST surface. The kill switch is bound to loopback only.

Method	Path	Role
GET	/api/options/screener	screener state + ranked rows
POST	/api/options/scan	start a screener scan
GET	/api/options/chain?symbol=	nearest non-ODTE chain with greeks
GET	/api/options/engine	engine state: positions, journal, config
POST	/api/options/engine/arm	arm / disarm (separate from enabled)
POST	/api/options/engine/flatten	sell all open options, cancel working buys
POST	/api/options/engine/config	patch engine config knobs
GET	/api/agent/state	agent + supervisor state
POST	/api/agent/chat	LLM chat with tools
GET	/api/brain/state	guard-skill status + brain journal
POST	/api/kill	loopback-only kill switch: disarm + flatten

Appendix C — Glossary

Table 16. Terms as used in this report.

Term	Meaning
SIP	Securities Information Processor — the consolidated equity tape (CTA/UTP)
OPRA	Options Price Reporting Authority — the consolidated options feed
indicative feed	Alpaca's free options data tier: derived/delayed values, not actionable quotes
0DTE	zero days to expiry — same-day-expiring contract
spread %	$(\text{ask} - \text{bid}) / \text{mid} \times 100$ — this report's spread measure
mark-to-bid	valuing a long position at the bid — the price you could exit at
fill class	ledger classification of a fill vs the quote: at_bid / at_ask / through_bid ...
DCA layer	add-on entry into an existing position (pyramiding)
chase	re-pegging an unfilled limit order toward the market
RiskGate	the mandatory pre-trade checkpoint (Chapter 6)
guard skill	a named deterministic monitor inside AgentBrain with bounded actions
wanted-set	the minimal symbol set the data hub subscribes to

Appendix D — The Public Face

The project ships with a public landing page (convexity.faruqueamin.com) whose 'Live demo' button opens the actual command desk — positions, gates, journal and brain state streaming over WebSocket. Source is at github.com/faruqueamin/convexity. The desk figure in this report is a stylized render of that console; the live version is one click away.

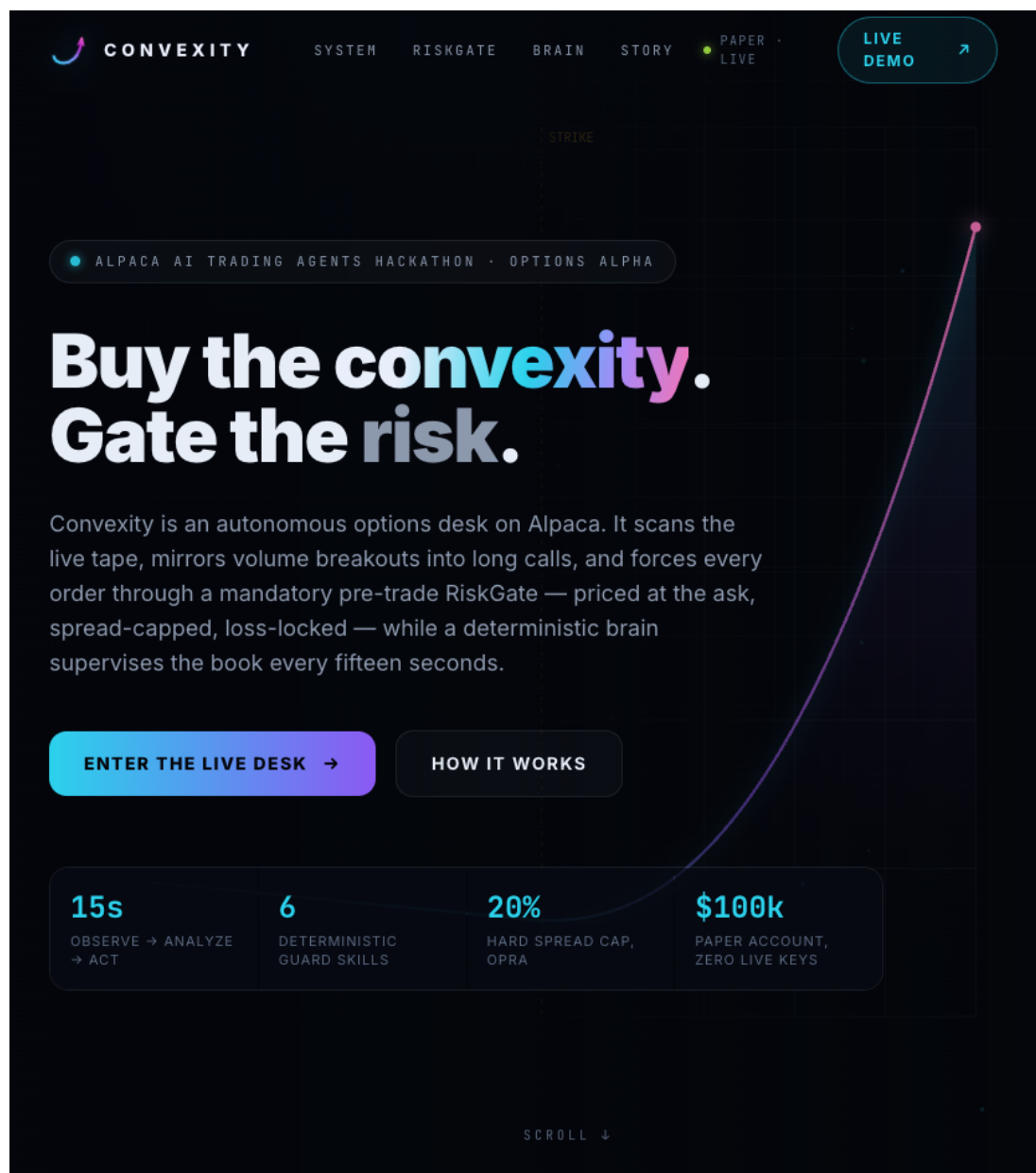


Figure 12. The landing page hero — the convex payoff motif is the system's thesis in one image.

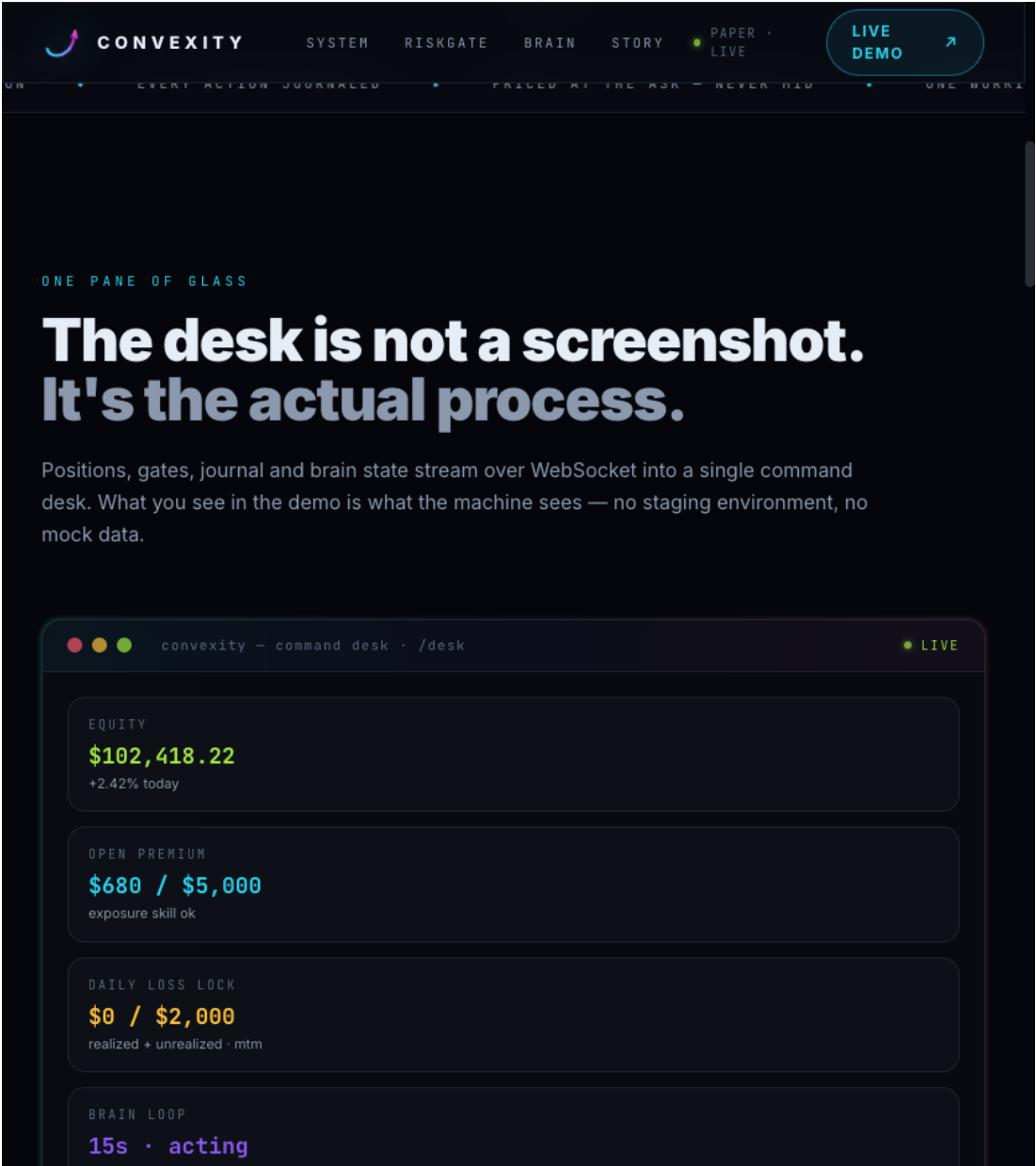


Figure 13. Landing-page render of the command desk: equity, exposure, loss lock and brain loop panels above a live journal excerpt.

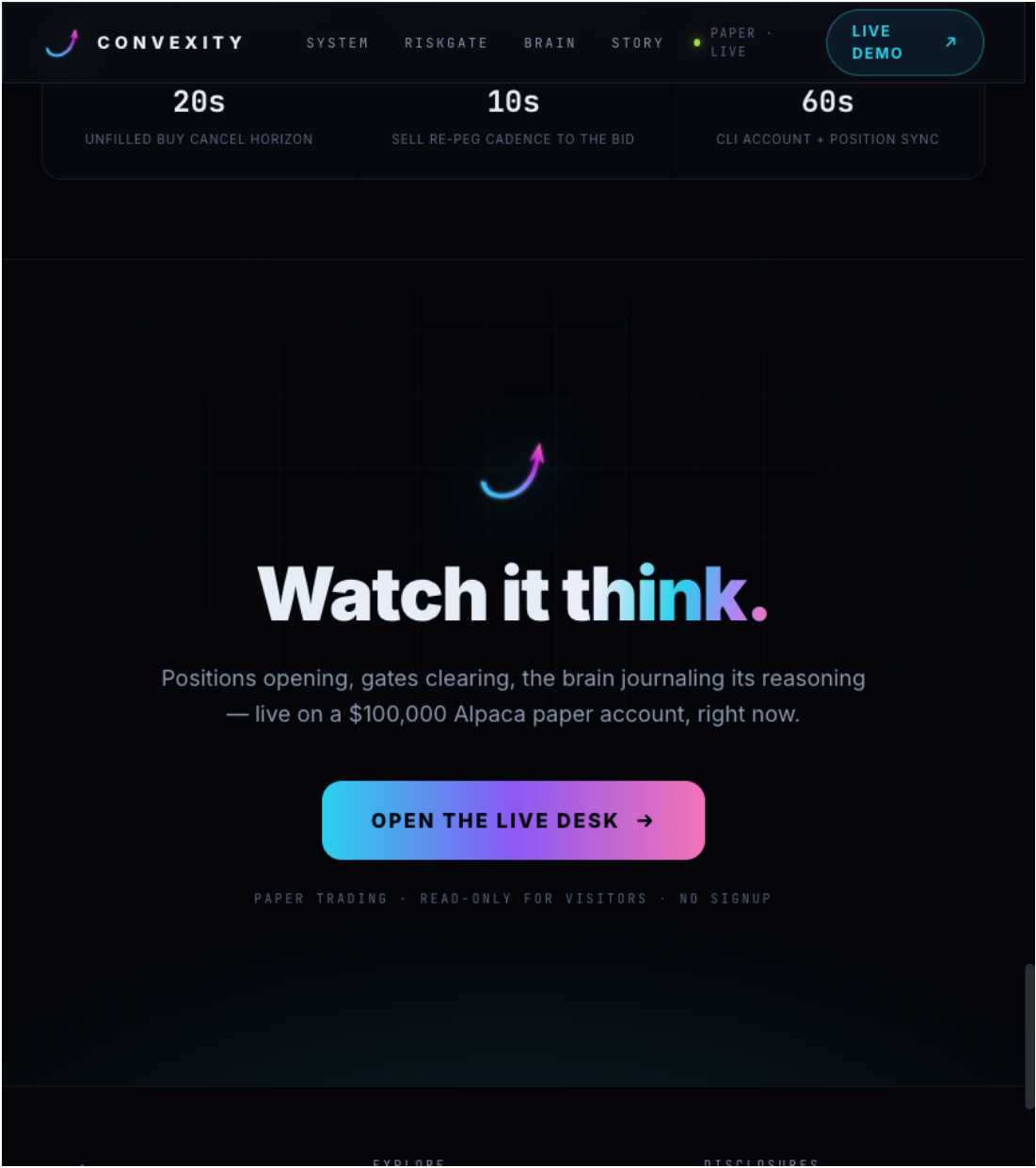


Figure 14. Closing call-to-action — the live desk is paper trading, read-only for visitors, and requires no signup.

Appendix E — Reproduction Guide

Every figure and number in this report can be regenerated from the repository. The process boots disarmed; nothing trades without two deliberate switches.

```
# 1. Run the desk (paper-only; keys go in envs/vol10s-paper.env)
npm install
pm2 start ecosystem.config.js          # http://127.0.0.1:8977/desk

# 2. Optional agent layer
ollama pull gemma3:12b

# 3. Inspect the artifacts this report is built on
data/vol10s-paper/fills.jsonl          # every order event, with quotes
data/vol10s-paper/brain-state.json     # the brain's journaled decisions
data/vol10s-paper/options-scan.json    # the persisted screener scan
docs/incident-2026-08-28.md            # the post-mortem

# 4. Regenerate figures + this PDF
python docs/thesis/make_figs.py
python docs/thesis/build_thesis.py
```

The landing page build is equally reproducible: `cd landing && npm run deploy` rebuilds the React site into the server's public directory, where it is served at the subdomain root.

Appendix F — Repository Map

```

optionalpaca/
■ ■ src/
■ ■ ■ vol10s-paper-server.js      # process entry: wires every service
■ ■ ■ services/
■ ■ ■ ■ vol10s/
■ ■ ■ ■ ■ options/RiskGate.js    # Chapter 6 – the mandatory gate
■ ■ ■ ■ ■ OptionsPlayEngine.js  # Chapter 7 – entries, exit stack
■ ■ ■ ■ ■ LiveMarketHub.js      # Chapter 4 – the flood funnel
■ ■ ■ ■ ■ OptionScreener.js     # §7.4 – universe ranking
■ ■ ■ ■ ■ PaperFillLledger.js    # the ledger behind every figure
■ ■ ■ ■ ■ Vol10sPaperServer.js   # REST + WebSocket + static hosting
■ ■ ■ ■ agent/
■ ■ ■ ■ ■ AgentBrain.js         # Chapter 8 – guard skills
■ ■ ■ ■ ■ AgentService.js       # LLM prompt-protocol tool loop
■ ■ ■ ■ ■ AgentSupervisor.js    # advisory-first risk reviews
■ ■ docs/
■ ■ ■ incident-2026-08-28.md     # the post-mortem (Chapter 5)
■ ■ ■ hackathon-writeup.md      # submission write-up
■ ■ ■ thesis/                   # this document + figure generators
■ ■ ■ landing/                  # React/Vite public landing page
■ ■ ■ data/vol10s-paper/        # journals, ledger, scan, state

```

Appendix G — Fill Ledger Field Dictionary

Every order event appends one JSON line to `data/vol10s-paper/fills.jsonl`. The schema is the report's evidence base; it is documented here so the ledger can be mined by anyone reproducing the analysis.

Table 17. The ledger schema. The pricing fields exist because Chapter 3's lesson was learned the expensive way: if an event's quote state isn't recorded, its cost can't be reconstructed.

Field	Meaning
ts / epoch	event time (see §4.4: post-incident rows carry true epoch ms)
event	submit · replace · cancel · fill
kind / side	equity option · buy sell
symbol / occ	underlying, and the OCC contract code for options
qty / filledQty	requested and filled contracts/shares
limitPx / fillPx	staged limit; actual fill price
peg	where the limit sat: bid · mid · ask
bid / mid / ask	the quote state at the event — the pricing-reality record
spreadPct	$(ask - bid) / mid \times 100$ at the event
feed	opra indicative — provenance of the quote
vsBid / vsMid / vsAsk	fill price relative to each reference
class	execution class: at_bid · at_ask · through_bid · ...
latencyMs	submit → fill wall time
cash / equity	account state snapshot at the event
dailyPnl / openPremium	engine risk counters at the event
orderId	broker order id — the dedupe key (one fill per order)

End of report. Convexity — buy the convexity, gate the risk. convexity.faruqueamin.com · github.com/faruqueamin/convexity